

On breaking McEliece keys using brute force

Lorenz Panny

Technische Universität München, Germany

lorenz@yx7.cc

Abstract. In the McEliece public-key encryption scheme, a private key is almost always not determined uniquely by its associated public key. We highlight a structural characterization of equivalent private keys that reduces the cost estimate for a simple private-key search using the support-splitting algorithm (SSA) by a polynomial but practically very substantial factor. In addition, we show how to apply the attack to extended codes in order to further improve the performance of the attack. (All of these techniques appear to be known to experts, but not all details have previously been laid out in the literature.)

In addition to spelling out the — thus far — missing details underlying these attack strategies, we provide an optimized software implementation of the SSA for this kind of key search and demonstrate its capabilities in practice by solving a key-recovery challenge with a naïve a-priori cost estimate of 2^{91} bit operations in just ≈ 1470 core days, testing ≈ 7700 private-key candidates per core and second in the process. We stress that the speedup from those equivalences on private keys and from our implementation techniques is merely polynomial and does not indicate any weakness in realistic instantiations of the McEliece cryptosystem, whose parameter choices are primarily constrained by decoding attacks rather than ludicrously more expensive key-recovery attacks.

Keywords: Code-based cryptography · concrete cryptanalysis · Goppa codes

1 Introduction

Code-based cryptography, and McEliece’s public-key encryption scheme [McE78] in particular, are one of the primary candidates for post-quantum cryptography. The idea is simple: Decoding is computationally hard for *random* linear codes, whereas there exist efficient decoders for some more structured families of codes. Users can thus generate an efficiently decodable code in private and “scramble” it into an equivalent code which “appears” random and can be published. (Concretely, for the secret code, the standard choice is a Goppa code over the binary finite field $\mathbb{F}_2$, which can be decoded efficiently using algorithms due to Patterson [Pat75] or Bernstein [Ber24].) The secret transformation from the structured private

* Date of this document: 2026-01-30.

code to the public random-looking code thus forms a trapdoor that allows the private-key holder to decode efficiently, while (presumably) nobody else will be able to do so. As a consequence, as pointed out in McEliece’s original paper:

It appears that an eavesdropper has two basic attacks to try; first, to try to recover G from G' and so to be able to use Patterson’s algorithm. Second, he might attempt to recover $\mathbf{u}$ from $\mathbf{x}$ without learning G . [McE78]

These two approaches are nowadays known as the “key-recovery attack” and the “decoding attack” against McEliece, respectively. For decoding, the state of the art is *information-set decoding (ISD)*, of which there exist a large number of variants: See [CME22r, § 1] for a long list of references. In comparison, the key-recovery problem has been receiving relatively slim consideration in the literature. The reason for this seems to be that parameter choices for McEliece have always been constrained primarily by the decoding attack, which has a much lower complexity than all known key-recovery attacks. Indeed, as pointed out by McEliece:

The first attack seems hopeless if n and t are large enough because there are so many possibilities for G , not to mention the possibilities for S and P . [McE78]

However, for independent applications that might rely on the hardness of the McEliece key-recovery problem only, and in the interest of gaining sufficient clarity about the situation in general, there still seems to be value in determining the exact difficulty of this problem.

In this paper, we theoretically investigate and practically optimize a well-known approach for the key-recovery problem, which falls into the category of “smart brute force”: Enumerate part of the private-key components (the Goppa polynomial g and *set* of evaluation points $\{\alpha_1, \dots, \alpha_n\}$) and check in an efficient manner, using Sendrier’s *support-splitting algorithm* [Sen00], whether there exists a valid assignment for the remaining component (the permutation of evaluation points) that leads the key-generation routine to produce the targeted public key.

The historical lack of concrete cost estimates for the McEliece key-recovery problem prompted the *Technology Innovation Institute (TII)* in Abu Dhabi to issue a set of challenge instances for this problem, covering estimated difficulties between 22 and 255 bits. The largest challenge instance that was solved during the admissible time span (May 2023 to May 2024) was a McEliece public key with parameters $(m, t, n) = (8, 5, 253)$ and an a-priori difficulty estimate of 83 bits. This estimate matches the result of multiplying the number of private keys up to permutation ($\approx 2^{tm}/t \cdot \binom{2^m}{n}$) by a rough cost estimate for Gauß elimination (n^3), corresponding to a straightforward black-box application of the support-splitting algorithm to the key-recovery problem while setting constant factors in the cost of linear algebra to one. We demonstrate that this naïve estimate for the complexity of a brute-force key search overshoots the actual cost in two orthogonal ways: First, the *number* of guesses required is smaller (by a logarithmic factor) than the number of private keys, since a polynomially large set of private keys corresponds to any given public key. Second, the cost of testing each candidate private key can be *amortized* across multiple guesses, resulting in a lower cost per guess

compared to testing each guess completely separately and independently. We further describe and demonstrate some useful implementation techniques which drastically speed up the key search in practice.

After the conclusion of the challenge, our implementation was further improved by incorporating the option to run the attack using extended Goppa codes, which introduces additional equivalences on private-key data, thereby accelerating the attack by another factor of $\approx n$. This renders it feasible to solve the McEliece key-recovery problem for even larger parameter sets: The biggest example we have conquered to date is TII’s challenge instance with an a-priori difficulty estimate of 91 bits, which took less than a (wall-clock) day on a potent server.

Acknowledgements. This work was provoked by the *TII McEliece Challenges*.¹ Thanks to Andre Esser for encouraging me to delve into this topic, to Violetta Weger for educating me about some aspects of coding theory, and to Marius Stelzner for pointing out an assortment of small mistakes in a previous version. I also wish to thank my previous employer, Academia Sinica, whose equipment was used during part of this research.

2 Preliminaries

2.1 Definitions and notation

The following definitions are all standard in coding theory: A *(linear) code* in $\mathbb{F}_q^n$ is a subspace $C \subseteq \mathbb{F}_q^n$. The *(Hamming) weight* of a vector $c \in \mathbb{F}_q^n$ is the number of nonzero coordinates in c . The *enumerator* $\mathcal{W}(C)$ of a linear code $C \subseteq \mathbb{F}_q^n$ is the vector $(w_0, w_1, \dots, w_n) \in \mathbb{Z}_{\geq 0}^{n+1}$ where $w_i = |\{c \in C \mid \text{wt}(c) = i\}|$. The *dual* of a code C is the code $C^\perp = \{w \in \mathbb{F}_q^n \mid \forall v \in C. \langle v, w \rangle = 0\}$. The *hull* of C is the code $C \cap C^\perp$. Two codes are *permutation-equivalent* if one can be obtained from the other by permuting the coordinates of the ambient space $\mathbb{F}_q^n$.

Throughout, let $\text{lrr}_t(q)$ denote the number of monic irreducible polynomials over $\mathbb{F}_q$ of degree t . Note $q^t - \log_2(t) \cdot q^{t/2} \leq t \cdot \text{lrr}_t(q) \leq q^t$, hence $\text{lrr}_t(q) \approx q^t/t$.

We write $\mathbb{F}_q[x]_{\leq t}$ for the set of polynomials in $\mathbb{F}_q[x]$ of degree $\leq t$; similarly for “ $<$ ” in place of “ $\leq$ ”.

2.2 Goppa codes

Fix a prime power $q = p^m$ and let t, n be positive integers such that $tm \leq n \leq q$. For a monic squarefree polynomial $g \in \mathbb{F}_q[x]$ of degree t and a length- n sequence $L = (\alpha_1, \dots, \alpha_n)$ of distinct elements $\alpha_i \in \mathbb{F}_q$ satisfying $g(\alpha_i) \neq 0$, the *Goppa code* defined by g and L is the $\mathbb{F}_p$ -vector space

$$\Gamma(g, L) = \left\{ c \in \mathbb{F}_p^n \mid \forall f \in \mathbb{F}_q[x]_{< t}. \sum_{i=1}^n c_i (f/g)(\alpha_i) = 0 \right\}.$$

¹ <https://crowdchallenge.tii.ae/mceliece-challenges/>

Identifying elements of $\mathbb{F}_{p^m}$ with vectors in $\mathbb{F}_p^m$ using an arbitrary $\mathbb{F}_p$ -vector space isomorphism, a parity-check matrix of $\Gamma(g, L)$ is given by [MS81, Ch. 12, § 3]

$$\begin{bmatrix} \alpha_1^0/g(\alpha_1) & \cdots & \alpha_n^0/g(\alpha_n) \\ \alpha_1^1/g(\alpha_1) & \cdots & \alpha_n^1/g(\alpha_n) \\ \vdots & \ddots & \vdots \\ \alpha_1^{t-1}/g(\alpha_1) & \cdots & \alpha_n^{t-1}/g(\alpha_n) \end{bmatrix} \in \mathbb{F}_p^{tm \times n}.$$

Notice that permuting L evidently produces equivalent Goppa codes. We shall write $\text{set}(L)$ for the set $\{\alpha_1, \dots, \alpha_n\}$ containing all the elements in L .

Extended Goppa codes. We now additionally allow for the list of evaluation points to contain a new element ∞ , so that the list $\widehat{L}$ will consist of $n+1$ distinct elements of $\mathbb{P}^1\mathbb{F}_q = \mathbb{F}_q \cup \{\infty\}$. Note that we do not require $\infty \in \widehat{L}$; thus, the definition here is more general than the definition from [MS81, Ch. 12, § 4]. By convention, for a nonzero rational expression $f(x) = f_1(x)/f_2(x)$ with $f_1, f_2 \in \mathbb{F}_q[x]$, we set $f(\infty) = 0$ when $\deg(f_1) < \deg(f_2)$, $f(\infty) = \text{lc}(f_1)/\text{lc}(f_2)$ when $\deg(f_1) = \deg(f_2)$ and $f(\infty) = \infty$ when $\deg(f_1) > \deg(f_2)$.

With this, the *extended Goppa code* defined by g and $\widehat{L}$ is the $\mathbb{F}_p$ -vector space

$$\widehat{\Gamma}(g, \widehat{L}) = \left\{ c \in \mathbb{F}_p^{n+1} \mid \forall f \in \mathbb{F}_q[x]_{\leq t}, \sum_{i=1}^{n+1} c_i (f/g)(\alpha_i) = 0 \right\}.$$

The following lemma shows that this definition is indeed a generalization of [MS81, Ch. 12, § 4], where extended Goppa codes are constructed by “adding an overall parity check”:

Lemma 1. *If $\widehat{L}$ is obtained by extending a list $L = (\alpha_1, \dots, \alpha_n) \in \mathbb{F}_q^n$ with $\alpha_{n+1} = \infty$, i.e., $\widehat{L} = L \parallel (\infty)$, then $\widehat{\Gamma}(g, \widehat{L}) = \{c \mid (-c_1 - \dots - c_n) \mid c \in \Gamma(g, L)\}$.*

Proof. See [Appendix A.1](#).

Thus, for $\widehat{L} = L \parallel (\infty)$ in particular, a parity-check matrix of $\widehat{\Gamma}(g, \widehat{L})$ is given by

$$\begin{bmatrix} \alpha_1^0/g(\alpha_1) & \cdots & \alpha_{n+1}^0/g(\alpha_{n+1}) & 0 \\ \alpha_1^1/g(\alpha_1) & \cdots & \alpha_{n+1}^1/g(\alpha_{n+1}) & 0 \\ \vdots & \ddots & \vdots & \vdots \\ \alpha_1^{t-1}/g(\alpha_1) & \cdots & \alpha_{n+1}^{t-1}/g(\alpha_{n+1}) & 0 \\ 1 & \cdots & 1 & 1 \end{bmatrix} \in \mathbb{F}_p^{(tm+1) \times (n+1)}.$$

In general, the column $(0, 0, \dots, 0, 1)$ is associated to the element ∞ in the list $\widehat{L}$; if $\text{set}(\widehat{L}) \not\ni \infty$, no such column appears. It should therefore not come as a surprise that a Goppa code can be recovered from its associated extended Goppa code:

Corollary 2. *Let $g^{(1)}, g^{(2)} \in \mathbb{F}_q[x]$ be polynomials as in the definition of an (extended) Goppa code and let $L^{(1)}, L^{(2)}$ be lists of n distinct elements of $\mathbb{F}_q$ each. If $\widehat{\Gamma}(g^{(1)}, L^{(1)} \parallel (\infty)) = \widehat{\Gamma}(g^{(2)}, L^{(2)} \parallel (\infty))$, then $\Gamma(g^{(1)}, L^{(1)}) = \Gamma(g^{(2)}, L^{(2)})$.*

Proof. By [Lemma 1](#), the last coordinate of each code word that is “lost” when transitioning from $L^{(i)} \parallel (\infty)$ to $L^{(i)}$ contains entirely redundant information. $\square$

As a consequence of [Lemma 1](#), the extended Goppa code $\widehat{C} = \widehat{\Gamma}(g, L \parallel (\infty))$ associated to a Goppa code $C = \Gamma(g, L)$ can be constructed efficiently from C without knowledge of g or L , by adding an overall parity-check symbol to C . Together with [Corollary 2](#), this reduces solving permutation equivalence for two Goppa codes $C^{(1)}$ and $C^{(2)}$ to solving permutation equivalence for the associated *extended* Goppa codes $\widehat{C}^{(i)}$, provided that the permutation $\pi \in \mathcal{S}_n$ realizing the equivalence between $\widehat{C}^{(1)}$ and $\widehat{C}^{(2)}$ can be forced to map the positions of ∞ in the lists $\widehat{L}^{(i)}$ defining the two extended codes to one another. (We will see in [Section 3.1](#) that this is indeed possible.)

2.3 Key search

The most obvious approach to recovering a McEliece private key is to simply guess the pair (g, L) , compute the associated public key, and check for equality. Since there are $q!/(q-n)!$ choices for L , it is easily seen that $\text{Irr}_t(q) \cdot q!/(q-n)!$ attempts suffice on average. (This is under the assumption that g is irreducible.)

One may gain a reduction factor of $n!$ in the number of attempts by guessing $\text{set}(L)$ instead of L , and consequently checking for permutation equivalence of the resulting codes rather than equality. This test can be performed efficiently in practice using the *support-splitting algorithm* [\[Sen00\]](#). Using this approach, the expected number of required guesses appears to be $\text{Irr}_t(q) \cdot \binom{q}{n}$, although testing each guess now requires slightly more computation than before.

However, as it turns out, the actual complexity of a brute-force key search is even lower than this: The reason is that every public key has a relatively large number of equivalent private keys associated to it, and recovering any equivalent key suffices; indeed, finding *the* private key is impossible since it is information-theoretically indistinguishable from its equivalent keys. We will analyze these additional equivalences in the upcoming [Section 3](#).

Using extended Goppa codes. Furthermore, it has previously been pointed out that *extending* the Goppa code under attack yields a speedup for the attack: This observation is briefly mentioned, albeit without any details, in [\[LS01, § 2.2.1\]](#) and [\[OS09, p. 126\]](#). We will elaborate on this approach in [Section 3.1](#).

3 Equivalent keys

In this section, we study an important algebraic relationship on the data defining a Goppa code which induces equivalences on the resulting codes. The protagonist is the following group of transformations:

Definition 3. The affine semilinear group of a finite field $\mathbb{F}_q$ is the subgroup

$$\text{AGL}(q) := \left\{ (x \mapsto Ax^\varphi + B) \mid A \in \mathbb{F}_q^\times, B \in \mathbb{F}_q, \varphi \in \text{Aut}(\mathbb{F}_q) \right\}$$

of $\text{Sym}(\mathbb{F}_q)$. Its elements are called affine semilinear transformations.

One subtle detail in [Definition 3](#) is that elements of $\text{AGL}(q)$ might be viewed either as a triple (A, B, φ) or as a permutation in $\text{Sym}(\mathbb{F}_q)$. It is easy to see that these two viewpoints are indeed equivalent:

Lemma 4. Consider the triples (A, B, φ) and (A', B', φ') with $A, A' \in \mathbb{F}_q^\times$, $B, B' \in \mathbb{F}_q$, and $\varphi, \varphi' \in \text{Aut}(\mathbb{F}_q)$.

If $Ax^\varphi + B = A'x^{\varphi'} + B'$ holds for all $x \in \mathbb{F}_q$, then $(A, B, \varphi) = (A', B', \varphi')$.

Proof. See [Appendix A.2](#).

Corollary 5. The cardinality of $\text{AGL}(q)$ equals $(q-1)qm$ where $m = \log_p(q)$.

Remark 6. Explicitly, in the “triples” viewpoint, formulas for computing in $\text{AGL}(q)$ are given as follows:

$$\begin{aligned} (A, B, \varphi)^{-1} &= ((1/A)^{\varphi^{-1}}, (-B/A)^{\varphi^{-1}}, \varphi^{-1}); \\ (A', B', \varphi') \circ (A, B, \varphi) &= (A'A^{\varphi'}, A'B^{\varphi'} + B', \varphi' \circ \varphi). \end{aligned}$$

The group $\text{AGL}(q)$ evidently acts on $\mathbb{F}_q$. We extend this action to subsets of $\mathbb{F}_q$ by element-wise application and to vectors in $\mathbb{F}_q^n$ by coordinate-wise application. Furthermore, we define an action on polynomials in $\mathbb{F}_q[x]$ by acting on the roots:

Definition 7. Let $h \in \mathbb{F}_q[x]$ denote a polynomial of degree d and $\varrho_1, \dots, \varrho_d$ its (not necessarily distinct) roots in $\overline{\mathbb{F}_q}$. For $\tau = (x \mapsto Ax^\varphi + B) \in \text{AGL}(q)$, lift φ to an arbitrary automorphism $\Phi \in \text{Aut}(\overline{\mathbb{F}_q})$ which restricts to φ in $\text{Aut}(\mathbb{F}_q)$,² and define $\tau * h$ as the polynomial $\text{lc}(h) \cdot \prod_{j=1}^d (x - A\varrho_j^\Phi - B) \in \mathbb{F}_q[x]$.

Lemma 8. The group action from [Definition 7](#) is well-defined.

Proof. See [Appendix A.3](#).

The following main theorem is a generalization of a result originally due to Moreno [\[Mor79\]](#), which was restricted to the important special case of irreducible g and $\text{set}(L) = \mathbb{F}_q$, in characteristic 2 only. The cryptographic literature usually attributes Moreno’s result to Gibson [\[Gib91\]](#), who appears to have rediscovered it later and introduced it to the cryptanalytic context. The generalization given here is essentially due to [\[Ber00\]](#), who declares it to be “well known”:

² The obvious way to do this is to lift $\varphi: \mathbb{F}_q \rightarrow \mathbb{F}_q, x \mapsto x^{p^k}$ to $\Phi: \overline{\mathbb{F}_q} \rightarrow \overline{\mathbb{F}_q}, x \mapsto x^{p^k}$.

Theorem 9. *Let g and L be as in [Section 2.2](#) and let $\tau \in \text{AFL}(q)$. Then*

$$\Gamma(\tau * g, \tau * L) = \Gamma(g, L) .$$

Proof. This is proved in [\[Ber00, Proposition 3\]](#).

Regrettably, however, the group action by $\text{AFL}(q)$ on the set of McEliece private keys does not immediately lead to either lower *or* upper bounds on the number of equivalent private keys per public key. This is due to two separate, counteracting effects:

- It can occasionally happen that some of the equivalences induced by $\text{AFL}(q)$ as in the theorem are *already* explained by permutation equivalence: This is the case whenever $\text{AFL}(q)$ stabilizes the polynomial g and acts as a permutation on the list L , i.e., stabilizes $\text{set}(L)$. As a consequence, simply multiplying the number of equivalences coming from $\text{AFL}(q)$ by the cardinality $n!$ of $\mathcal{S}_n$ leads to overcounting the number of equivalent private keys in general.
- Not all equivalences on public keys are explained by the action of $\text{AFL}(q)$: We refer to this phenomenon as “spurious equivalences”. They, too, are rare, but they may in general lead to undercounting the number of equivalent private keys when estimating it via $\text{AFL}(q)$.

It seems difficult to strictly prove anything about the prevalence and impact of those effects for a given choice of parameters.

From a practical standpoint, however, since both of these exceptional cases occur only exceedingly rarely when concerned with reasonably large parameter sets, we may for our cryptanalytic purposes assume that we are in the average situation, in which the equivalent keys are indeed in bijection with $\text{AFL}(q)$ and the count resulting from [Heuristic 10](#) is therefore essentially (or exactly) correct.

Heuristic 10. *Generically, the number of irreducible Goppa codes with parameters $p, m, q = p^m, t, n$ modulo permutation equivalence can be estimated as*

$$\text{Irr}_q(t) \cdot \binom{q}{n} / ((q-1)qm) .$$

Also note that public keys having more equivalent private keys are more likely to be sampled during key generation, hence the distribution of public keys is biased towards the “weaker” ones with respect to a brute-force key-search attack.

3.1 Equivalences between extended Goppa codes

The following results are known to experts and have appeared before in [\[Ber00\]](#), but do not appear to have experienced widespread recognition in the cryptography community.

Definition 11. The projective semilinear group of $\mathbb{F}_q$ is the subgroup

$$\text{P}\Gamma\text{L}(q) := \left\{ \left(x \mapsto \frac{Ax^\varphi + B}{Cx^\varphi + D} \right) \mid A, B, C, D \in \mathbb{F}_q, \det \begin{pmatrix} A & B \\ C & D \end{pmatrix} \neq 0, \varphi \in \text{Aut}(\mathbb{F}_q) \right\}$$

of $\text{Sym}(\mathbb{P}^1\mathbb{F}_q)$, where by convention $1/0 = \infty$ and $\infty \mapsto A/C$. Its elements are called projective semilinear transformations.

Lemma 12. The cardinality of $\text{P}\Gamma\text{L}(q)$ equals $(q^2 - 1)qm$ where $m = \log_p(q)$.

Proof. See [Appendix A.4](#).

As before, we extend the action of $\text{P}\Gamma\text{L}(q)$ on $\mathbb{P}^1\mathbb{F}_q$ to subsets of $\mathbb{P}^1\mathbb{F}_q$ by element-wise application, to vectors in $(\mathbb{P}^1\mathbb{F}_q)^{n+1}$ by coordinate-wise application, and to polynomials $g \in \mathbb{F}_q[x]$ with no roots in $\mathbb{F}_q$ by acting on the roots (analogously to [Definition 7](#) and [Lemma 8](#)).

Theorem 13. Let g and $\hat{L}$ be as in [Section 2.2](#), with the additional restriction that g have no roots in $\mathbb{F}_q$, and let $\tau \in \text{P}\Gamma\text{L}(q)$. Then

$$\hat{\Gamma}(\tau * g, \tau * \hat{L}) = \hat{\Gamma}(g, \hat{L}).$$

Proof. This is proved in [\[Ber00, Theorem 1\]](#). □

Just like for the action of $\text{A}\Gamma\text{L}(q)$, this yields an estimate for the number of equivalence classes of extended Goppa codes:

Heuristic 14. Generically, the number of extended irreducible Goppa codes with parameters $p, m, q = p^m, t, n$ modulo permutation equivalence can be estimated as

$$\text{irr}_q(t) \cdot \binom{q+1}{n+1} / ((q^2 - 1)qm).$$

The ratio between the quantities from [Heuristic 14](#) and [Heuristic 10](#) equals $\binom{q+1}{n+1} / (q+1) / \binom{q}{n} = 1/(n+1)$, which translates to a speedup factor of about $n+1$ in the attack when using extended codes. (In TII's McEliece Challenges, $n \approx q$.)

Restricting equivalences between extended Goppa codes. The main application of the action by $\text{P}\Gamma\text{L}(q)$ on extended codes is that there are more equivalences between Goppa codes after extending than there are between the original Goppa codes, hence the probability of finding parameters (g, L) such that the resulting extended Goppa code is equivalent to a given extended Goppa code increases by a factor of approximately $n+1$ compared to when not extending the codes (according to [Heuristics 10](#) and [14](#)).

The strategy is as follows: Given a Goppa code $C = \Gamma(g, L)$ with unknown parameters g, L , we extend C by adding an overall parity check, yielding the extended Goppa code $\hat{C} = \hat{\Gamma}(g, L \parallel (\infty))$. Using a brute-force search over $(g', \hat{L}')$ and a solver for permutation equivalence, we obtain an extended Goppa code $\hat{C}' = \hat{\Gamma}(g', \hat{L}')$ which is permutation-equivalent to $\hat{C}$. For ease of notation, we view such an equivalence as a bijection $\hat{\pi}: \text{set}(L \parallel (\infty)) \rightarrow \text{set}(\hat{L}')$.

The “lucky” case is $\infty \in \widehat{L}'$ and $\widehat{\pi}(\infty) = \infty$, since in this case we can immediately invoke [Corollary 2](#) to conclude that $C' = \Gamma(g', L')$, where L' is $\widehat{L}'$ with ∞ removed, defines a code equivalent to $\Gamma(g, L) = C$, so (g', L') is a valid solution to our key-recovery problem.

Otherwise, we may again exploit the action of $\text{PFL}(q)$ on the pair $(g', \widehat{L}')$ to position ourselves in the “lucky” case: In essence, one looks for some $\tau \in \text{PFL}(q)$ that maps the given element $\widehat{\pi}(\infty) \in \mathbb{F}_q$ to ∞ . Then, using [Theorem 13](#), the pair $g'' := \tau * g'$ and $\widehat{L}'' := \tau * \widehat{L}'$ still satisfies $\widehat{\Gamma}(g'', \widehat{L}'') = \widehat{C}'$, which is still permutation-equivalent to $\widehat{C}$, but now with a permutation $\text{set}(L \parallel (\infty)) \rightarrow \text{set}(L'')$ that fixes ∞ , thus leaving us in the “lucky” case.

Finding a $\tau \in \text{PFL}(q)$ that takes a given $\alpha \in \mathbb{F}_q$ to ∞ is of course easy enough: One such example is $\tau = (x \mapsto \frac{1}{x-\alpha})$.

4 The support-splitting algorithm

In the year 2000, Sendrier introduced a practically efficient algorithm for testing whether two linear codes are permutation-equivalent, and if so, to recover the permutation [[Sen00](#)]. The approach crucially relies on *permutation invariants* of linear codes: Required are efficiently computable values associated to a linear code which are (1) invariant under permutations, and (2) discriminant, i.e., they tend to take on different values for codes which aren’t permutation-equivalent.

We remark that not much has been proven about either the correctness or the complexity of this algorithm. Empirically, however, it appears to work very well for random codes as well as for the family of Goppa codes used in McEliece.

Splitting the support. Provided a sufficiently discriminant invariant $\mathcal{V}$, given two codes $C, C' \subseteq \mathbb{F}_q^n$, it is conceptually easy to recover a permutation $\pi \in \mathcal{S}_n$ connecting the two codes, assuming one exists: The core subroutine takes as input two partitions $\{1, \dots, n\} = S_1 \dot{\cup} \dots \dot{\cup} S_\ell = T_1 \dot{\cup} \dots \dot{\cup} T_\ell$ such that $\pi(S_i) = T_i$ for all $i \in \{1, \dots, \ell\}$, and outputs another, finer pair of partitions with the same property. Repeating this process will eventually yield a partition with all $|S_i| = |T_i| = 1$, which allows us to read off π . (If the codes are in fact not permutation-equivalent, the subroutine should either detect this and fail, or output an arbitrary finer partition: One may simply check at the end that π is a valid solution.)

We say that the partitions $S_1 \dot{\cup} \dots \dot{\cup} S_\ell$ and $T_1 \dot{\cup} \dots \dot{\cup} T_\ell$ are “*compatible*” if $|S_i| = |T_i|$ holds for all $i \in \{1, \dots, \ell\}$, i.e., if the cardinalities of each pair of sets in the partitions with the same index match up.

Initial partition. The refining step below assumes that a relatively fine partition of the set of positions of C and C' has already been computed. Obtaining such an initial partition can be done easily by puncturing both codes C and C' at each position $i \in \{1, \dots, n\}$ and computing the invariants $\mathcal{V}$ associated to each

punctured code C_i resp. C'_i : The partition is then given by grouping all i with the same $\mathcal{V}(C_i)$ together into one S_k , and similarly for $\mathcal{V}(C'_i)$ into T_k .

If $\mathcal{V}(C) \neq \mathcal{V}(C')$, or if the partitions for C and C' resulting from the initial puncturings are not compatible, the codes cannot be permutation-equivalent.

Refining step. The refining procedure works as follows: Iterate over nonempty subsets $I \subseteq \{1, \dots, \ell\}$, for instance in order of increasing cardinality up to some predetermined constant bound. For each such I , puncture C at the positions in $J := \bigcup_{i \in I} S_i$ to obtain a code C_J . Then compute the associated invariant $\mathcal{V}$ for all further puncturings $C_{J \cup \{j\}}$ at each position j . Now, hopefully, $\mathcal{V}$ is nonconstant on $\{C_{J \cup \{j\}} \mid j \in S_k\}$ for some S_k with $k \notin I$, so that we may refine the partition by splitting such a S_k into multiple sets $S_{k,1} \dot{\cup} \dots \dot{\cup} S_{k,r} = S_k$. Applying the same procedure to C' with the T_k leads to a partition $T_{k,1} \dot{\cup} \dots \dot{\cup} T_{k,r} = T_k$ for which $\pi(S_{k,i}) = T_{k,i}$. In this case, we have thus successfully refined the partition.

The refining step can fail in two ways:

- First, it could happen that there are no more subsets $I \subseteq \{1, \dots, \ell\}$ among those considered for which $\mathcal{V}$ refines the partition after puncturing at the positions indicated by I . In that case, one may resort to brute-forcing parts of the permutation π in order to refine the partition anyway—in a possibly incorrect way, but of course a valid guess must exist if the codes are actually permutation-equivalent, so a backtracking search will eventually succeed.
- Second, if the newly refined partitions are not compatible, then the codes cannot be permutation-equivalent.

Remark 15. The original description of the support-splitting algorithm [Sen00] also applies *shortenings* in addition to puncturings to the input codes. We found that this adds only little discriminatory power while significantly complicating the algorithms, hence we decided not to use shortenings in our implementation.

Hull enumerators. The key insight in [Sen00] was that the *enumerator* of the *hull* of a code forms a permutation invariant which is almost always efficiently computable and sufficiently discriminant, at least in practice, to enable the support-splitting algorithm to work well.

On average, hulls are random-looking subspaces of relatively small dimension: It was proven in [Sen97, Theorem 4] that the dimension of the hull for a random code roughly obeys a discrete half-normal distribution;³ hence, the probability of it exceeding a certain $\ell_0 \in \mathbb{Z}_{\geq 0}$ is bounded above by $q^{-\Omega(\ell_0^2)}$, which shrinks at a superexponential rate as ℓ_0 grows.

³ Asymptotically, the proportion of n -dimensional codes over $\mathbb{F}_q$ with hull dimension ℓ approaches $R_\ell := C / \prod_{i=1}^\ell (q^i - 1) \approx C / q^{\ell(\ell+1)/2}$, where C is a constant (dependent on q) which satisfies $0.419 < C < 1$. Note that $\sum_{\ell=\ell_0}^\infty R_\ell \in \Theta(R_{\ell_0})$.

As a consequence, computing the hull enumerator of a sufficiently random code can almost always be done very quickly: Since the dimension of the hull is overwhelmingly likely to be tiny, one may simply iterate over all hull vectors and tally their weights. Indeed, the core observation in [Sen00] was that the hull enumerator is (empirically) an efficiently computable and sufficiently discriminant invariant $\mathcal{V}$ to render the support-splitting approach highly effective in practice.

Optimizations. Implementing the support-splitting algorithm in a straightforward manner, using standard linear-algebra routines and hash tables for keeping track of the partitions S_k and T_k , leads to significant overheads from dynamic memory allocations and unpredictable memory-access patterns: See Section 5.

We foreshadow that there are two very useful optimizations for the support-splitting algorithm when applied to brute-force key search:

- First, noticing that almost all candidate codes are actually *not* equivalent to the target code, instead of implementing and running the full support-splitting algorithm inside the brute-force search, it is enough to build a simple and fast *filter* which will reject most inequivalent codes swiftly, but need not be perfectly correct, nor recover the permutation in full. See Section 5.5.
- Second, since one of the input codes remains fixed, one may precompute good puncturing locations by essentially running only one side of the support-splitting algorithm for the target code and noting which puncturings worked well to refine the partition, then performing only those puncturings for the candidate equivalent code at runtime while skipping the rest. See Section 5.6.

5 Efficient key search

In the context of a brute-force key search, in which checking for (permutation) equivalence contributes a multiplicative factor to the overall complexity, it is clearly desirable to minimize the cost per equivalence test.

To set a baseline, let us first discuss the main limitations of a straightforward implementation: Processing a single guess at a time (per thread), representing codes as generator matrices or parity-check matrices, and using generic linear-algebra libraries for the hull and punctured-hull computations. This approach is prone to suffering from several effects causing slowdown:

- Plenty of conditional branches, which may cause mispredictions and therefore expensive rollbacks and cache invalidations. This includes many conditions whose values are essentially (difficult-to-predict) coinflips, such as during the computation of a reduced row echelon form using Gauß elimination.
- Relatively complex, a priori variably-sized data structures. Depending on implementation specifics, these data structures may incur slowdowns in thread-safe memory management on many-core machines. In addition, for large data structures, another issue is that unpredictable memory-access patterns may cause the processor to stall while waiting for data to be fetched from main memory to a fast cache located closer to the processing core.

By contrast, in the following, we describe a collection of techniques designed to work around these issues on typical contemporary many-core CPUs. The key strategy employed in this work is to rephrase the entire equivalence test as a **binary circuit**, which is much more amenable to fast parallel evaluation, on simpler hardware, than the original high-level algorithm. As a byproduct, the same circuit can easily be ported to other platforms, such as GPUs, FPGAs, or even ASICs, to obtain efficient implementations of the key search.

Remark 16. It seems very much conceivable that generic circuit-optimization techniques could help reduce the number of logical operations in the circuits constructed via our approach, leading to a faster key-recovery attack on any of the aforementioned hardware platforms. We have not explored this possibility.

Remark 17. The main case of interest going forward will be binary codes ($q=2$). However, whenever the algorithms given in the following are easily expressed for general q , we have refrained from needlessly specializing to $q=2$. Throughout, we will identify the values $0, 1 \in \mathbb{F}_2$ with the booleans `false` and `true` respectively.

Moreover, note that some of the algorithms are formulated using (fixed-length) loops and branching for clarity: These abstractions are not admissible in a circuit to evaluate the algorithm and will have to be unrolled into copies of the loop body and conditional assignments using bitmasks, respectively. See the difference between [Algorithm 1](#) and [Algorithm 2](#) for the latter conversion.

The algorithmic building blocks given in [Sections 5.2](#) and [5.3](#) are essentially from [[Sen00](#), §6.1]. We reiterate them here for clarity and completeness, and because some of our optimizations ([Section 6.1](#)) interact with those details.

5.1 Preliminaries

Throughout the rest of the paper, we adhere to the following definitions and notational conventions.

- A q -ary *code* of dimension m and length n is an m -dimensional subspace of $\mathbb{F}_q^n$. Codes are represented using a *basis matrix* in $\mathbb{F}_q^{m \times n}$, whose rows form an $\mathbb{F}_q$ -basis of the code. Occasionally, basis matrices may be padded (or interleaved) with zero rows.
- The *dual* code $C^\perp$ associated to a code C is its orthogonal complement with respect to the standard inner product. It equals the right kernel of a basis matrix. The *hull* of a code C is the intersection $C \cap C^\perp$.
- *Puncturing* a code $C \subseteq \mathbb{F}_q^n$ at position $i \in \{1, \dots, n\}$ means projecting it to the subspace $\mathbb{F}_q^{i-1} \times \{0\} \times \mathbb{F}_q^{n-i}$. On a basis matrix, this can be realized by filling the i^{th} column with zeroes.
- The *weight* $\text{wt}(c) \in \{0, \dots, n\}$ of a code word $c \in \mathbb{F}_q^n$ is the number of nonzero coordinates in c . The (*weight*) *enumerator* $\mathcal{W}(C)$ of a code $C \subseteq \mathbb{F}_q^n$ is the integer vector $(w_0, \dots, w_n) \in \mathbb{Z}_{\geq 0}^{n+1}$, where $w_i = |\{c \in C \mid \text{wt}(c) = i\}|$.

5.2 Keeping pivots on the diagonal

A useful trick to increase the amount of deterministic code paths in the linear-algebra routines is to ensure all pivots are kept on the main diagonal inside the row echelon form computation, by inserting zero rows between non-zero rows in the matrix as appropriate. (This is a non-standard choice: Most descriptions of Gauß elimination keep “empty” rows at the end.) See [Figure 1](#) for an example.

$$\begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Figure 1. Generating matrices of the same 7-dimensional subspace of $\mathbb{F}_2^{10}$ given in standard reduced row echelon form (left) and in the “diagonal standard form” (right) that is employed in [\[Sen00\]](#) and in this work.

Definition 18. A square matrix M is in diagonal standard form if its nonzero rows are in reduced row echelon form and all pivots lie on the main diagonal.

This form of a matrix is very convenient for the linear-algebraic types of computational tasks that appear in coding theory: In particular, having a generator matrix of some code in this form allows us to directly read off a basis of the right kernel of the matrix, i.e., a generator matrix of the dual code. The following result was used without proof in [\[Sen00, §6.1.1\]](#):

Lemma 19. A matrix M in diagonal standard form is idempotent, i.e., $M^2 = M$. In particular, the nonzero columns of $\mathbb{1} - M$ form a basis of the right kernel of M .

Proof. See [Appendix A.5](#).

Using the diagonal standard form was already suggested in the original description of the support-splitting algorithm; however, the motivation there seems to have been to enable [Algorithm 4](#) for punctured hulls given in the following [Section 5.3](#), rather than to optimize the circuit associated to the linear-algebra computation. Indeed, the diagonal standard form (and a transformation matrix) can be computed using a variant of Gauß elimination that involves fewer case distinctions based on the locations of the pivots; see [Algorithms 1](#) and [2](#).

Lemma 20. [Algorithm 2](#) is correct and requires $6n^3 + O(n^2)$ logical operations. If the output L is not needed, omitting its computation reduces this to $2n^3 + O(n^2)$.

Proof. Several routine verifications. (See also the comments in [Algorithm 2](#).) $\square$

Algorithm 1: Transforming a matrix to diagonal standard form.

– Restartable: If the k^{th} column is replaced, continue the main loop from that k .

Input: $A, B \in \mathbb{F}_q^{n \times n}$.

Output: $L, R \in \mathbb{F}_q^{n \times n}$ such that R is in diagonal standard form, and such that there exists $T \in \text{GL}_n(\mathbb{F}_q)$ with $L = TA$ and $R = TB$.

// Initialize output variables.

$(L, R) \leftarrow (A, B)$

// Outer loop: Iterate over columns from left to right.

for k **from** 1 **to** n **do**

if $R_{kk} = 0$ **then**

 // We don't have a pivot. Try to find one.

for i **from** 1 **to** n **do**

if $i = k$ **then**

continue

 // Does column k not yet have a pivot in row k , but in row i ?

if $R_{kk} = 0$ **and** $(i \geq k \text{ or } R_{ii} = 0)$ **and** $R_{ik} \neq 0$ **then**

$L_k \leftarrow L_k + L_i$

$R_k \leftarrow R_k + R_i$

break

if $R_{kk} = 0$ **then**

continue

 // Rescale row k row so that the pivot equals 1.

$L_k \leftarrow R_{kk}^{-1} \cdot L_k$

$R_k \leftarrow R_{kk}^{-1} \cdot R_k$

 // Inner loop: Zero out the rest of column k .

for i **from** 1 **to** n **do**

if $i = k$ **then**

continue

if $R_{ik} = 0$ **then**

continue

$L_i \leftarrow L_i - R_{ik} \cdot L_k$

$R_i \leftarrow R_i - R_{ik} \cdot R_k$

return (L, R)

Algorithm 2: Transforming a matrix over $\mathbb{F}_2$ to diagonal standard form.

- Constant-time: There are no input-dependent branches or memory accesses.
- Restartable: If the k^{th} column is replaced, continue the main loop from that k .
- Note: The output usually differs from the one produced by [Algorithm 1](#).

Input: $A, B \in \mathbb{F}_2^{n \times n}$.

Output: $L, R \in \mathbb{F}_2^{n \times n}$ such that R is in diagonal standard form, and such that there exists $T \in \text{GL}_n(\mathbb{F}_2)$ with $L = TA$ and $R = TB$.

// Initialize output variables.

$(L, R) \leftarrow (A, B)$

// Outer loop: Iterate over columns from left to right.

for k **from** 1 **to** n **do**

// Inner loop: Try to find a row with a pivot and add it to the correct row.

for i **from** 1 **to** n **do**

if $i = k$ **then**

└ **continue**

// Does column k not yet have a pivot in row k , but in row i ?

$m \leftarrow R_{kk}$

if $i < k$ **then**

└ $m \leftarrow m \vee R_{ii}$

$m \leftarrow \neg m$

$m \leftarrow m \wedge R_{ik}$

// If so, add row i to row k .

for j **from** 1 **to** n **do**

└ $L_{kj} \leftarrow L_{kj} \oplus (m \wedge L_{ij})$

for j **from** k **to** n **do**

└ $R_{kj} \leftarrow R_{kj} \oplus (m \wedge R_{ij})$

// Inner loop: Try to zero out the rest of column k assuming we have a pivot.

for i **from** 1 **to** n **do**

if $i = k$ **then**

└ **continue**

// Does column k contain a one in row i ?

$m \leftarrow R_{ik}$

// If so, add row k to row i .

for j **from** 1 **to** n **do**

└ $L_{ij} \leftarrow L_{ij} \oplus (m \wedge L_{kj})$

for j **from** k **to** n **do**

└ $R_{ij} \leftarrow R_{ij} \oplus (m \wedge R_{kj})$

return (L, R)

From the given formulation of [Algorithm 2](#), it is straightforward to construct a binary circuit that is equivalent to evaluating the entire algorithm: See [Figure 3](#) in [Appendix B](#) for a tiny example. Note that this is just for illustration: We do not actually ever compute or export a graph representation of the circuit for the fast SSA filter at any point; rather, we write software (very similar to [Algorithm 2](#)) to evaluate the circuit straight away. See also [Section 6.1](#).

Another important property of [Algorithms 1](#) and [2](#) is that they are *restartable*: When some of the columns of the input matrix are substituted, the execution of the algorithm can continue from the new columns and thereby reuse all the work done up until the first modified column, resulting in a much lower amortized cost. This is very useful in the context of McEliece key recovery, which involves a brute-force search over subsets of possible column vectors. See [Section 6.2](#) and also [\[BLP08, § 4\]](#), where a very similar approach was used for the decoding attack.

Remark 21. When applied in the context of [Algorithm 3](#), the input matrix B to [Algorithm 2](#) is symmetric. This causes the transformation matrix T to become symmetric as well (it equals $(B + D)^{-1}$ where D is the diagonal part of $\mathbb{1} - R$), but unfortunately, we have not succeeded in exploiting this fact to eliminate redundancy and accelerate the algorithm.

Remark 22. Some common optimizations for linear algebra over small finite fields, such as the “Four Russians” method [\[ADKF70\]](#), unfortunately do not apply in our setting: This is because it relies on fast random access to precomputed tables, which becomes expensive in the circuit abstraction.

5.3 Computing everywhere punctured hulls, all at once

All of the following is essentially explained in [\[Sen00, § 6.1.2\]](#). In this section, we give a simplified and streamlined account for our particular application.

Lemma 23. *Let C be a code given by a square matrix $M \in \mathbb{F}_q^{n \times n}$ in diagonal standard form. Then the hull $C \cap C^\perp$ equals the (right) kernel of $\mathbb{1} + M - M^\top$.*

Proof. This follows from the general fact that $(C_1 \cap C_2)^\perp = C_1^\perp + C_2^\perp$ holds for any two codes C_1, C_2 , combined with [Lemma 19](#). $\square$

What this entails is that computing the hull of a linear code can be done entirely *in place*: The nonzero rows of $\mathbb{1} - M$ correspond exactly to the zero rows of M , hence computing the hull via [Lemma 23](#) involves flipping all coefficients lying above the diagonal across the diagonal, negating them, and filling the diagonal with ones, then computing the diagonal standard form again and reading off the kernel. See [Algorithm 3](#).

In addition, [Lemma 23](#) also allows for computing the hulls of *punctured* codes very easily after the hull has been computed: This was detailed for the case $q = 2$ in [\[Sen00, § 5.3.1\]](#). See [Algorithm 4](#).

Algorithm 3: Computing the hull of a code in diagonal standard form.

Input: $M \in \mathbb{F}_q^{n \times n}$ in diagonal standard form, defining a code $C \subseteq \mathbb{F}_q^n$.
Output: $H \in \mathbb{F}_q^{\ell \times n}$ a basis matrix of the hull $C \cap C^\perp$.
 $A \leftarrow 0 \in \mathbb{F}_q^{n \times n}$
 $B \leftarrow M$
// Iterate over all rows.
for k **from** 1 **to** n **do**
 // If this row is zero, copy in the corresponding row of $(\mathbb{1} - M)^\top$.
 if $M_{kk} = 0$ **then**
 $A_{kk}, B_{kk} \leftarrow 1$
 for j **from** 1 **to** $k - 1$ **do**
 $A_{kj}, B_{kj} \leftarrow -M_{jk}$
 // Compute the diagonal standard form of B .
Apply **Algorithm 1** to the input (A, B) , obtaining matrices $L, R \in \mathbb{F}_q^{n \times n}$.
// Read off a basis of the hull from the nonzero columns of $(\mathbb{1} - R)^\top$.
Initialize $\ell \leftarrow 0$ and $H \in \mathbb{F}_q^{0 \times n}$.
for k **from** 1 **to** n **do**
 if $R_{kk} = 0$ **then**
 Append a zero row to H .
 $\ell \leftarrow \ell + 1$
 $H_{\ell k} \leftarrow 1$
 for i **from** 1 **to** $k - 1$ **do**
 $H_{\ell i} \leftarrow -R_{ik}$
return H

Algorithm 4: Computing the hull of a singly-punctured code (over $\mathbb{F}_2$).

Input: $H \in \mathbb{F}_2^{\ell \times n}$ a basis matrix of the hull of a code $C \subseteq \mathbb{F}_2^n$,
the matrix $L \in \mathbb{F}_2^{n \times n}$ as computed by **Algorithm 3** for C ,
an index $r \in \{1, \dots, n\}$.
Output: $H' \in \mathbb{F}_2^{\ell' \times n}$ a basis matrix of the hull of the code C punctured at r .
 $H' \leftarrow H$
// If r is in the support, shorten the code at r .
for k **from** ℓ **down to** 1 **do**
 if H_{kr} **then**
 for i **from** 1 **to** $k - 1$ **do**
 if H_{ir} **then**
 $H'_i \leftarrow H_i \oplus H_k$
 Delete the k^{th} row from H' .
 return H'
// Otherwise, the hull is either augmented by a suitable vector, or unchanged.
if $L_{r,r} = 0$ **then**
 Append the row L_r of L to H' .
return H'

Remark 24. The algorithms in this paper only require singly-punctured hull enumerators. However, if further puncturings are required, this could easily be achieved using the same methods by keeping track of the matrices L, R computed by [Algorithm 1](#) over the course of multiple sequential applications of [Algorithm 4](#).

5.4 Computing weight enumerators

Computing the weight enumerator of a low-dimensional code over a small finite base field is generally straightforward: Simply enumerate all linear combinations of a basis of the code and record the weight of each code word encountered this way. An optimized method of performing this enumeration (for $q = 2$) consists in iterating over the vectors determining linear combinations of a basis according to a *Gray code*, i.e., a way of arranging all bit vectors of a given length in such a way that every pair of two adjacent vectors has Hamming distance one. The details are elaborated in [Algorithm 5](#).

Each individual count inside a weight-enumerator data structure is represented in our binary circuit as a standard ripple-carry incremter of some suitable (a priori fixed) bit length w . Every such counter also features an overflow bit o to prevent further processing of wrong results after an integer wraparound occurred. See [Algorithm 6](#). We note that the counts could alternatively be stored in unary representation, but this scales much worse: Incrementing a w -bit binary number takes $2w + O(1)$ bit operations, while incrementing a unary number capable of storing w -bit integers takes $2^w + O(1)$ bit operations.

Algorithm 5: Weight enumerator of a low-dimensional code (over $\mathbb{F}_2$).

Input: $H \in \mathbb{F}_2^{\ell \times n}$ a basis matrix of some code in $\mathbb{F}_2^n$.
Output: Weight enumerator $(w_0, \dots, w_n) \in \mathbb{Z}_{\geq 0}^{n+1}$ of the code generated by H .
Initialize counters $w_0, \dots, w_n$ to zero.
Initialize $c \leftarrow 0 \in \mathbb{F}_2^n$.
Increment w_0 .
for s **from** 1 **to** $2^\ell - 1$ **do**
 $i \leftarrow \nu_2(s)$ // The number of trailing zero bits.
 $c \leftarrow c \oplus H_{i+1}$
 // Compute the weight of c .
 Initialize a counter k to zero.
 for j **from** 1 **to** n **do**
 if c_j **then**
 Increment k .
 // Increment the associated counter.
 for j **from** 1 **to** n **do**
 if $k = j$ **then**
 Increment w_j .
return $(w_0, \dots, w_n)$

Algorithm 6: Incrementing a w -bit integer using bit operations.

Input: Bit array $[b_0, \dots, b_{w-1}; o]$ representing $n = b_0 + 2b_1 + \dots + 2^{w-1}b_{w-1}$
if $o = \text{false}$, else $n = \infty$, and a bit c .

Output: Bit array $[b'_0, \dots, b'_{w-1}; o']$ representing $n' = n + c$ if $n + c < 2^w$,
else $n' = \infty$.

for i **from** 0 **to** $w - 1$ **do**

$b'_i \leftarrow b_i \oplus c$
 $c \leftarrow b_i \wedge c$

$o' \leftarrow o \vee c$

return $[b'_0, \dots, b'_{w-1}; o']$

5.5 Fast filtering using hull enumerators

In the context of a key-recovery attack against McEliece, the main contributing factor of the attack complexity is the cost of repeated permutation-equivalence tests with one of the two inputs (the target public key) remaining fixed. As such, our main goal is to optimize those equivalence tests as much as possible, yielding a “fast filter”. Crucially, the construction of the fast filter itself may utilize liberal amounts of precomputation involving the target public-key code.

The key observation is that (1) for codes to possibly be equivalent, the hull enumerators and the multisets of singly-punctured hull enumerators of the two codes must be identical, and (2) it is acceptable for this equivalence test to be imperfect, opening up the potential to achieve much faster execution times by admitting a small chance of false positives.

In the context of our attack, we successfully exploit these two observations to obtain a fast (but imperfect) equivalence check as follows:

- **Puncturings only.** While the original description of the support-splitting code applies both puncturings and shortenings to the input codes, we refrain from using shortenings for simplicity. (See [Remark 15](#).)
- **Single puncturings only.** The full support-splitting algorithm involves a recursive procedure to further discriminate the positions of the two input codes corresponding to each other until the exact permutation inducing the equivalence has been determined. To trade speed for correctness, we instead stop after the first refining step and post-process the resulting stream of candidate equivalent codes using a slower but perfectly correct implementation of the full support-splitting algorithm only afterwards. (See [Section 5.7](#).)
- **Sets instead of multisets.** In theory, we would like to compare the multisets of singly-punctured hull enumerators of an input code to the (precomputed) multiset corresponding to the target public-key code. In reality, this multiset comparison can be approximated by instead simply checking for the presence or absence of certain elements of one multiset in the other multiset: As soon as an expected hull enumerator cannot be found, or an unexpected hull enumerator is encountered, the respective code can immediately be discarded.

The resulting fast filtering method is detailed in [Algorithm 7](#). It relies on the following data structure as input:

Definition 25. For a code $C \subseteq \mathbb{F}_q^n$, a fast filter is the following data structure:

- The first component is the hull enumerator of C .
- The second component is a list of tuples $(b, v) \in \{\text{true}, \text{false}\} \times \mathbb{Z}_{\geq 0}^{n+1}$ with the property that $b = \text{true}$ if and only if the vector v occurs as one (or multiple) of the singly-punctured hull enumerators of C .

In a nutshell, this data structure contains information about singly-punctured hull enumerators whose presence or absence quickly establishes (with certainty) the non-equivalence of “most” random codes to the target code: Check for the presence of enumerators which appear for the target code C but are unlikely to appear in a random code, and check for the absence of enumerators which are likely to appear for a random code but do not appear for C .

Algorithm 7: Evaluating a fast filter for permutation equivalence.

Input: Fast filter $\mathcal{F} = (u, [(b_1, v_1), \dots, (b_\ell, v_\ell)])$, some code $C' \subseteq \mathbb{F}_q^n$.

Output: Boolean indicating whether C' passed the filter $\mathcal{F}$ or not.

// First make sure the hull enumerators match.

Compute the hull enumerator u' of C' .

// [Algorithms 3](#) and [5](#)

if $u' \neq u$ **then**

return false

// Verify that all forbidden singly-punctured hull enumerators are absent.

for k **from** 1 **to** n **do**

 Compute the hull enumerator v'_k of C' punctured at k . // [Algorithms 4](#) and [5](#)

for i **from** 1 **to** ℓ **do**

if $b_i = \text{false}$ **and** $v'_k = v_i$ **then**

return false

// Verify that all required singly-punctured hull enumerators are present.

for i **from** 1 **to** ℓ **do**

if $b_i = \text{true}$ **then**

$r \leftarrow \text{false}$

for k **from** 1 **to** n **do**

if $v'_k = v_i$ **then**

$r \leftarrow \text{true}$

break

if $r = \text{false}$ **then**

return false

// All checks passed!

return true

The way a “fast filter” is constructed for a given target code C is simple: We sample many random Goppa codes, compute all their singly-punctured hull

enumerators, and use a greedy approach to select those enumerators into the filter first whose presence or absence leads to the strongest filtering. See [Algorithm 8](#).

Algorithm 8: Constructing a fast filter for permutation equivalence.

Input: Goppa code $C \subseteq \mathbb{F}_p^n$ with $q = p^m$ and $t = \deg(g)$; sample count $K \in \mathbb{Z}_{\geq 1}$.

Output: Fast filter $\mathcal{F}$ which C passes, whereas a random Goppa code with the same parameters is “unlikely” to pass (depending on K).

Compute the hull enumerator $u \in \mathbb{Z}_{\geq 0}^{n+1}$ of C .

Compute the set of all singly-punctured hull enumerators $V_{yes} \subseteq \mathbb{Z}_{\geq 0}^{n+1}$ of C .

for k **from** 1 **to** K **do**

 Sample a random Goppa code C'_k of parameters (p, m, t, n) .

 Compute the set of all singly-punctured hull enumerators $V'_k \subseteq \mathbb{Z}_{\geq 0}^{n+1}$ of C'_k .

$\mathcal{V} \leftarrow V_{yes} \cup \bigcup_{k=1}^K V'_k$

$\mathcal{C} \leftarrow \{C'_1, \dots, C'_K\}$

$\mathcal{F} \leftarrow (u, [])$

while $\mathcal{V} \neq \emptyset$ **do**

 Filter $\mathcal{C}$ through $\mathcal{F}$ and only keep the codes that pass.

 // At this point one could optionally **break** early once the empirical rate $|\mathcal{C}|/K$ of false positives has fallen below a given target false-positive rate.

if $\mathcal{C} = \emptyset$ **then**

break

for each $v \in \mathcal{V}$ **do**

$b_v \leftarrow (v \in V_{yes})$

 Compute the proportion ε_v of codes in $\mathcal{C}$ passing the filter $(u, [(b_v, v)])$.

 Find a $v \in \mathcal{V}$ with ε_v minimal among all v and remove v from $\mathcal{V}$.

 Append (b_v, v) to the second component of $\mathcal{F}$.

return $\mathcal{F}$

Remark 26. The “fast filtering” approach inherently offers a tradeoff between the complexity (hence the computational effort) of the employed strategy and the quality of the filtering (hence the rate of false positives).

In particular, with the greedy approach used in [Algorithm 8](#), a fast filter can simply be truncated at the end in order to increase the filter’s throughput at the expense of some of its distinguishing strength.

Remark 27. One side effect of the particular “fast filtering” technique employed in this work is that the key search proceeds at a much higher rate for some target codes than others: For example, in the most extreme case, the hull enumerator of the public-key code itself would be overwhelmingly unlikely to appear for random Goppa codes, hence a fast filter could consist of simply checking the hull enumerator (without applying any puncturings at all). This effect is visible in [Tables 1](#) and [2](#), where smaller “ $\# \mathcal{F}$ ” correlates with higher throughput overall.

The fact that one of the input codes remains fixed implies that bounds on the integers that can appear in weight enumerators, as needed for the algorithms

in [Section 5.4](#), are readily available: They can simply be chosen according to the weight enumerators observed for the target public key. While the enumerators computed for a candidate equivalent code may in fact have larger entries, this cannot possibly occur for a valid solution, hence this scenario can be detected (and the offending codes discarded) by simply inspecting the overflow bit.

5.6 Precomputation for the full SSA

For general inputs, the simple filter obtained in the previous [Section 5.5](#), which checks for the presence or absence of certain enumerators within the set of singly-punctured hull enumerators, may not be selective enough. In that case, one could proceed with precomputing the full SSA as described in [Section 4](#): Instead of relying only on singly-punctured hulls, continue down the recursion and puncture out more positions of the code as identified by their punctured hull enumerators.

An important detail is that this can be done as a one-time precomputation for one side of the equivalence test, leaving only the candidate equivalent key to be processed at runtime (following a predictable pattern determined beforehand), which is beneficial in the context of a key-recovery attack: It again enables the whole equivalence test to be phrased as a binary circuit that does not involve any unpredictable control flow or complicated data structures.

We also note that there is a tradeoff between the one-time precomputation cost and the online phase of the attack: Trying (in the notation of [Section 4](#)) a larger variety of puncturing sets I at each refining step yields possibly much shorter and therefore more efficient strategies for the resulting equivalence test.

In our implementation of the key-recovery attack (see [Section 6](#)), we initially tested a proof-of-concept implementation of this more general approach for the first fast filtering stage, but it quickly became evident that singly-punctured hulls almost always provide strong enough filtering in practice, hence the full “precomputed SSA” approach was abandoned for that purpose and we settled for the simpler strategy described in [Section 5.5](#). We do however use the more general method outlined here when post-processing the codes that passed the fast filter, as described in the following [Section 5.7](#).

5.7 Post-processing codes passing the filter

The fast filter from [Section 5.5](#) is not perfect: It does allow for false positives, i.e., actually inequivalent codes which nevertheless exhibit the correct set of singly-punctured hull enumerators, to be incorrectly identified as candidate equivalent codes — albeit at a very slow rate, assuming the conditions tested by the fast filter are strong enough, meaning they are satisfied only for a miniscule proportion of random Goppa codes.

Since those false positives are very rare in practice, a second, strictly correct (but more expensive) filtering stage can eliminate them at almost no additional cost per guess: We simply re-run the standard support-splitting algorithm (with a

precomputed strategy — see [Section 5.6](#)) in order to distinguish the true solution from the false positives that are erroneously passing through the filter. As a byproduct of establishing equivalence to the target code, the SSA at the same time also recovers a permutation that realizes the equivalence between the public-key code and the solution to the key-recovery problem.

6 Implementation

Our parallelized C++ implementation of the McEliece key-recovery attack using the support-splitting algorithm is available for download at the following link:

<https://yx7.cc/code/goppify/goppify-latest.tar.xz>

In this section, we specialize to $p = 2$ and $q = 2^m$; this is the only case that is currently supported by the implementation. Let w denote the number of bits in a register of the target architecture. (The implementation hardcodes $w = 64$.)

6.1 Bitslicing the fast SSA filter

The most effective optimization employed in our brute-force software to recover McEliece keys is *bitslicing*. Generically, this technique refers to rewriting a given computation as a binary circuit, then evaluating the circuit on w independent inputs *in parallel* using the processor’s bitwise logic instructions on registers holding w bits each. This is fundamentally the same concept as batching multiple instances of the same computation using vector instructions, but here it is applied at the level of single bits held in general-purpose CPU registers. See [Figure 2](#).

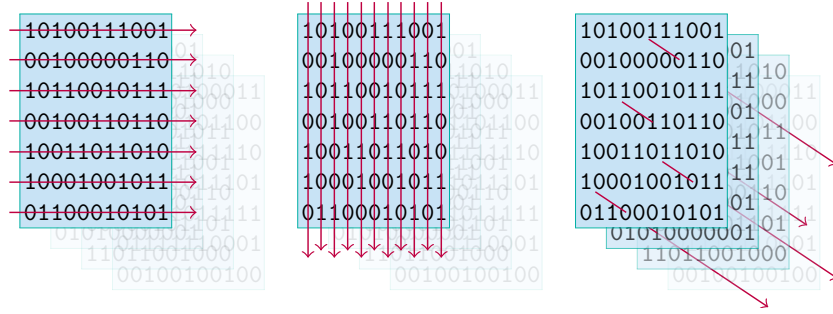


Figure 2. Illustration of the ways a list of matrices over $\mathbb{F}_2$ can be stored in, in (from left to right) row-major, column-major, and bitsliced order. Each arrow indicates the set of bits stored inside a single bit vector (i.e., CPU register) within the target machine.

In the context of linear algebra over $\mathbb{F}_2$, which the support-splitting algorithm heavily relies on, this yields to several important optimizations over the straightforward approach working on one matrix in row-major form at a time:

- **Predictable execution flow:** Avoiding conditional branches and variable-length loops assists the processor in effectively pipelining the execution of the program and helps prevent expensive mispredictions (and the resulting rollbacks) on architectures that rely on speculative execution.
- **Predictable memory-access pattern:** Avoiding random accesses to large memory regions is a crucial consideration in optimizing computations for modern CPUs since fetching data from main memory incurs a potentially very large time penalty. Using data-independent memory-access patterns enables the memory controller on superscalar architectures to prefetch the required information into a lower (faster) level of the cache hierarchy while previous steps of the program are in the process of being executed, resulting in much smaller memory-access latencies and, therefore, higher throughput.

On the opposite side, the primary drawbacks of bitslicing computations are:

- **Random accesses are expensive:** While loading or storing data from or to an array with variable index is expressible in the circuit abstraction (for an example, see [Remark 28](#) and the pivoting step in [Algorithm 2](#)), it comes at a linear time cost in the size of the array that is being accessed. In comparison, the RAM model of computation stipulates that a memory access can be performed in constant time, independently of the size of the memory, while more realistic models of computation assign a nonconstant but still sublinear time cost.
- **Integer arithmetic is expensive:** Modern processors typically perform a very large number of bit operations per assembly instruction or clock cycle. Logical bit operations, as employed for bitslicing, offer among the worst throughput in terms of bit operations per unit of time. By comparison, integer arithmetic in non-bitsliced representation benefits from specialized silicon inside the CPU, enabling it to perform integer operations with throughput often significantly greater than one instruction per cycle, despite the fact that those operations actually consist of a possibly very large number of bit operations. Therefore, emulating nontrivial computations using relatively primitive operations which could — in non-bitsliced representation — be run through specialized hardware is inherently much slower than simply using the built-in instructions. Nevertheless, when only a small number of simple (say) integer operations is required as part of a larger circuit that can more favorably be expressed in terms of bit operations, it may be beneficial to perform them in the bitsliced representation anyway.

Luckily, neither of those disadvantages has a big impact in our context: The vast majority of the time is spent on linear algebra over $\mathbb{F}_2$, where packing multiple instances into a set of registers in order to exploit the parallelism inherently provided by register-sized bitwise operators offers a clear advantage over using those same hardware units to execute only a single instance at a time.

Remark 28. Algorithms 1 to 7 and 9 are all given in non-bitsliced form for ease of notation. In all cases, converting them into a bitslicable circuit is straightforward: The most important translation step is to replace all conditional expressions by equivalent sequences of bit operations, similar to constant-time cryptographic code: In essence, the conditional assignment “**if** C **then** $R \leftarrow A$ **else** $R \leftarrow B$ ” turns into the computation of *both* branches $R_{\text{true}} \leftarrow A$ and $R_{\text{false}} \leftarrow B$, followed by the combination of the two possible results using a bitmask, i.e., computing $R \leftarrow (C \wedge R_{\text{true}}) \vee (\neg C \wedge R_{\text{false}})$. Similarly, random accesses to memory can be emulated by iterating over all memory cells that could possibly be accessed and conditionally reading or writing the cell using a bitmask each time.

Shortcuts. Recall that our implementation processes multiple instances of the hull and punctured-hull computations in parallel via bitslicing. For example, when *all* bits in the register holding the value m in Algorithm 2 are unset, the following loops have no effect and can thus be skipped altogether in bitsliced or vectorized software, resulting in noticeable speedups when only a small proportion of the input instances really do require the evaluation of that code path. We refer to this as a *shortcut*. Note that this breaks the circuit abstraction.

6.2 Orchestrating the key recovery

On top of the “fast filter” core for rapid batch testing of code equivalences using the bitsliced representation, our key-recovery attack tool consists of several more higher-level procedures that will be detailed below.

Our implementation of the algorithms described in this paper is written in C++23, making extensive use of its efficient and convenient multithreading functionality. We employ the NTL library for computing in $\mathbb{F}_{2^m}$ and $\mathbb{F}_{2^m}[x]$ outside of the bitsliced representation (i.e., for the computations involving $\mathbb{F}_{2^m}$ in Algorithm 10). Note that we exclusively work with parity-check matrices for manipulating Goppa codes, which are slightly easier (or at least more convenient) to construct than generator matrices.

The extended-codes version. For simplicity, in the following, we describe the version of the main algorithm that does not use extended Goppa codes. The only changes necessary to run the extended-code version are:

- Run the precomputation for the fast filter (Algorithm 8) for the extended version of the target public-key code instead of the target code itself.
- Instead of brute-forcing combinations of Goppa parity-check columns as below, brute-force combinations of *extended*-Goppa parity-check columns, which may or may not include a column $(0, \dots, 0, 1)$ associated to $\infty \in \text{set}(\widehat{L})$.
- Add a post-processing step to obtain a solution for the McEliece key-recovery problem given a solution for the key-recovery problem on extended codes: See Section 3.1.

The main loop. Until a solution is found, each processing core repeatedly samples w random (irreducible) polynomials $g_1, \dots, g_w \in \mathbb{F}_{2^m}[x]$ of degree t . For each $\alpha \in \mathbb{F}_{2^m}$, it computes the associated columns

$$v_\alpha^{(k)} := \begin{bmatrix} \alpha^0/g_k(\alpha) \\ \alpha^1/g_k(\alpha) \\ \vdots \\ \alpha^{t-1}/g_k(\alpha) \end{bmatrix} \in \mathbb{F}_2^{tm} \quad (*)$$

of a Goppa parity-check matrix (see [Section 2.2](#)). The collection of $2^m \cdot w$ columns $\{v_\alpha^{(k)} \mid k \in \{1, \dots, w\}, \alpha \in \mathbb{F}_{2^m}\}$ is then bitsliced into $2^m \cdot tm$ width- w registers $\{r_i(\alpha) \mid i \in \{1, \dots, tm\}, \alpha \in \mathbb{F}_{2^m}\}$, with $r_i(\alpha)$ containing the i^{th} coefficient in each of the columns $v_\alpha^{(1)}, \dots, v_\alpha^{(w)} \in \mathbb{F}_2^{tm}$.

Finally, the algorithm picks a random ordering for the 2^m elements of $\mathbb{F}_{2^m}$ and thus for the bitsliced columns $(r_1(\alpha), \dots, r_{tm}(\alpha))$ that will be combined into candidate parity-check matrices. See [Algorithm 10](#) for details.

Iterating over length- n subsets of $\mathbb{F}_q$. For each choice of the columns $r_i(\alpha)$, the algorithm now enumerates a certain (configurable) number of length- n subsets of those columns, which corresponds to guessing the set of Goppa evaluation points $\text{set}(L)$. These subsets are enumerated in a *depth-first* fashion, systematically constructing all possible combinations of column sets in the candidate Goppa parity-check matrix by filling in the columns from left to right, recursing further to the right whenever possible, and only continuing with the next choice for the “current” column after the algorithm has run out of options for all columns to the right. This enumeration procedure can be implemented using a *stack* data structure; see [Algorithm 9](#) for details.

One major benefit of enumerating subsets of $\mathbb{F}_{2^m}$ in this particular order is that almost all of the time, modifications occur exclusively near the end of the candidate parity-check matrix, which means most of the work involved in the linear-algebra routines can be reused and therefore amortized across multiple guesses for $\text{set}(L)$. This important optimization owes crucially to the fact that the particular variant of Gauß elimination used in [Algorithm 2](#) finalizes the columns of the reduced matrix in diagonal standard form sequentially from left to right, which renders it possible to restart the elimination process midway and continue from whichever column has been swapped out. This is similar to the techniques described in [\[BLP08, § 4\]](#).

Running the filter. Once the bitsliced candidate parity-check matrices have been set up in the way outlined above, all that is left to do is to run the fast filter developed in [Section 5.5](#) on the bitsliced inputs in parallel. This yields a single register containing one bit per candidate parity-check matrix, indicating whether

Algorithm 9: Searching for a valid subset of column vectors.

Input: Sequence $v_1, \dots, v_\ell$ of column vectors in $\mathbb{F}_q^d$, dimension $n \in \{1, \dots, \ell\}$, and a function $\text{check}: \mathbb{F}_q^{d \times n} \rightarrow \{\text{false}, \text{true}\}$.

Output: Iterator over subsets $\{i_1, \dots, i_n\} \subseteq \{1, \dots, \ell\}$ satisfying $\text{check}(R) = 1$, where R is the diagonal standard form of the matrix $(v_{i_1} \mid \dots \mid v_{i_n})$.

Initialize an empty stack $\mathcal{S}$ holding values in $\{1, \dots, \ell\}^* \times \mathbb{F}_q^{d \times d} \times \mathbb{F}_q^{d \times n}$.

Push $(((), \mathbb{1}, \emptyset)$ onto $\mathcal{S}$.

while $\mathcal{S}$ is not empty **do**

Pop the top value $((i_1, \dots, i_k), L, R)$ from $\mathcal{S}$.

if $k = n$ **then**

/* At this point, rather than enumerating all $\binom{\ell}{n}$ subsets, one could increment a counter and return early once some limit was reached. */

if $\text{check}(R)$ **then**

yield $\{i_1, \dots, i_k\}$

$j \leftarrow i_k + 1$

// convention: $i_0 = 0$

if $j \leq \ell$ **then**

if $0 < k \leq n$ **then**

// Note on the stack where to continue after the recursive step.

$(L', R') \leftarrow (L, R)$

Replace the k^{th} column of R' by the vector $L \cdot v_j$.

Apply the k^{th} iteration of **Algorithm 1** or **2** to the pair (L', R') .

Push $((i_1, \dots, i_{k-1}, j), L', R')$ onto $\mathcal{S}$.

if $0 \leq k < n$ **then**

// Set up the stack for entering a recursive depth-first step next.

Replace the $(k+1)^{\text{th}}$ column of R by the vector $L \cdot v_j$.

Apply the $(k+1)^{\text{th}}$ iteration of **Algorithm 1** or **2** to the pair (L, R) .

Push $((i_1, \dots, i_k, j), L, R)$ onto $\mathcal{S}$.

the respective instance has passed the filter. For each parity-check matrix that passed the filter, we reconstruct the matrix in non-bitsliced representation and run a slower non-bitsliced implementation of the full SSA (see [Section 5.7](#)) to discover whether the code is truly permutation-equivalent to the target public-key code or a false positive has passed the fast filter.

Algorithm 10: The main key-recovery algorithm.

Input: Fast filter $\mathcal{F}$ for a Goppa code $C \subseteq \mathbb{F}_2^n$ with $q = 2^m$ and $t = \deg(g)$.

Output: Goppa polynomial g and list L of evaluation points defining C .

while true do

Sample w irreducible polynomials $g_1, \dots, g_w \in \mathbb{F}_{2^m}[x]$ of degree t .

Compute the associated columns $v_\alpha^{(1)}, \dots, v_\alpha^{(w)}$ according to [Equation \(*\)](#).

For each $\alpha \in \mathbb{F}_{2^m}$, bitslice the list $[v_\alpha^{(1)}, \dots, v_\alpha^{(w)}] \in (\mathbb{F}_2^{tm})^w$ of w independent columns into a list of length- w registers $r(\alpha) = [r_1(\alpha), \dots, r_{tm}(\alpha)] \in (\mathbb{F}_2^w)^{tm}$.

Sample a random bijection $\iota: \{1, \dots, 2^m\} \rightarrow \mathbb{F}_{2^m}$ and let

$S \leftarrow [r(\iota(1)), \dots, r(\iota(2^m))] \in ((\mathbb{F}_2^w)^{tm})^{2^m}$.

Execute the bitsliced version of [Algorithm 9](#) with inputs S and n , using the bitsliced version of [Algorithm 7](#) with input $\mathcal{F}$ for the check function.

for each index set $\{i_1, \dots, i_n\} \subseteq \{1, \dots, 2^m\}$ returned by [Algorithm 9](#)
for the k^{th} bit of the bitsliced computation **do**

Rebuild the corresponding Goppa code $C' := \Gamma(g_k, [\iota(i_1), \dots, \iota(i_n)])$.

Run the standard SSA on input (C, C') , yielding a result $\pi \in \mathcal{S}_n \cup \{\perp\}$.

if $\pi \neq \perp$ **then**

$L \leftarrow [\iota(i_{\pi(1)}), \dots, \iota(i_{\pi(n)})]$

return (g, L)

Remark 29. [Algorithm 10](#) can easily be adjusted — following essentially the same structure — to work with any other efficient implementation of the fast filter, such as CPU implementations using dedicated vector registers and instructions, GPU implementations, or hardware implementations on FPGAs or ASICs.

6.3 Estimates

An overview of the estimated cost of breaking various (smaller) challenge instances of the McEliece key-recovery problem using our implementation of the attack is displayed in [Tables 1](#) and [2](#) (including, resp. without, the use of extended codes).

Baseline: Linear algebra using M4RI. The M4RI library [\[M4RI\]](#) is a highly optimized implementation of the “Four Russians” [\[ADKF70\]](#) approach to linear algebra over $\mathbb{F}_2$. Comparing the performance of our bitsliced implementation of Gauß elimination ([Algorithm 2](#)) to M4RI reveals that our implementation achieves roughly twice the throughput of M4RI when computing reduced echelon forms of random square matrices over $\mathbb{F}_2$ of dimensions $n = 253$. This suggests

that the vectorized circuit-based approach employed in this paper is presumably superior when many instances of the same linear-algebra task can be batched, even when the relatively complicated hardware required to run M4RI is available.

6.4 Breaking the “83-bit” instance

In this section we report on details of the successful private-key recovery for the largest solved instance (estimated 83 bits of security) of TII’s McEliece Challenges. The target public-key file `pk_McEliece_83.txt`, along with the public-key files for all other difficulty levels, are available at https://github.com/ElenaKirshanova/tii_decoding_challenge/blob/main/public_keyRec/.

Solving the challenge was done by simply invoking the script `./solve.sh 83` from (a previous version of) the code package without any further modifications. This script first runs the precomputation stage for the target code as described in Sections 5.5 and 5.6, followed by the main key-recovery program (Algorithm 10).

In a stroke of luck, for the particular target public-key code from the challenge, the hull enumerator alone is already quite rare: It only occurs for one in about 3350 random codes. Adding just one additional check for the presence of a certain singly-punctured hull enumerator lowers this proportion below one in a million, resulting in a high rate of filtering for this target key, as indicated in Table 1.

In another stroke of luck, the private key was encountered significantly faster than expected: While Table 1 suggests an estimated $2^{40.08}$ guesses, our software recovered a valid private key for the challenge instance after only $2^{39.04}$ guesses using about 1735 core days over the course of about 3.4 wall-clock days.

The hardware platform used to solve the challenge was a server with two AMD EPYC 9754 processors (containing a total of 256 cores implementing the Zen 4c microarchitecture) with dynamic frequency scaling (base clock 2.25 GHz, boost clock up to 3.1 GHz) and hyperthreading (for a total of 512 threads) enabled. Note that these timings are not exactly comparable to the estimates given in Table 1 since dynamic frequency scaling and hyperthreading were enabled at the time of the record run while they are disabled in Table 1.

6.5 Breaking the “91-bit” instance

After the conclusion of TII’s McEliece Challenges, our attack implementation was strengthened by adding the extended-codes optimization laid out in this paper. The outcome is a speedup factor on the order of n , which not only reduces the cost for solving the 83-bit challenge instance from about 1400 core days to about 6.3 core days, but also puts even larger challenge instances easily within reach: In an example run, we’ve successfully solved the TII McEliece Challenge instance with an a-priori difficulty estimate of 91 bits using only 161 core days on the same hardware configuration that was used to generate Tables 1 and 2.

Table 1. Overview of the smaller parameter sets from the *TII McEliece Challenges* together with attack complexity estimates using our optimized key-search software *without* the use of extended Goppa codes. The empirical throughput (private-key guesses per core and second) was measured on an AMD EPYC 9754 processor (Zen 4c microarchitecture) running at 2.25 GHz with hyperthreading disabled.

The required number of guesses was estimated based on [Heuristic 10](#). The columns “ $\# \mathcal{F}$ ” and “ $\approx \Pr[\mathcal{F} \mapsto \text{true}]$ ” refer to the length of the used fast filter $\mathcal{F}$, not including the unpunctured hull enumerator itself, and the empirical probability for a random code to pass the filter $\mathcal{F}$, respectively. All filters were constructed by running [Algorithm 8](#) with $K = 100,000$. Note that no attempt was made to optimize the estimated total duration of the attack by adjusting the length of the filter.

instance	m	t	n	$\approx \# \text{ guesses}$	$\# \mathcal{F}$	$\approx \Pr[\mathcal{F} \mapsto \text{true}]$	guesses/(core · s)	$\approx \text{core time}$
41	5	2	27	$2^{14.30}$	5	$2^{-6.31}$	$2^{16.77}$	$\approx 0.2 \text{ s}$
43	5	2	26	$2^{16.47}$	6	$2^{-4.25}$	$2^{14.92}$	$2^{1.54} \text{ s} \approx 2.9 \text{ s}$
44	6	2	61	$2^{11.76}$	10	$2^{-10.76}$	$2^{16.84}$	$\approx 0.0 \text{ s}$
45	6	3	62	$2^{12.83}$	9	$2^{-5.53}$	$2^{13.77}$	$\approx 0.5 \text{ s}$
46	6	4	63	$2^{13.44}$	4	$2^{-13.43}$	$2^{17.49}$	$\approx 0.1 \text{ s}$
47	5	2	23	$2^{21.42}$	7	$2^{-6.48}$	$2^{17.43}$	$2^{3.99} \text{ s} \approx 15.9 \text{ s}$
48	6	2	60	$2^{15.69}$	4	$2^{-15.69}$	$2^{18.68}$	$\approx 0.1 \text{ s}$
49	5	2	21	$2^{23.62}$	8	$2^{-10.48}$	$2^{20.18}$	$2^{3.44} \text{ s} \approx 10.9 \text{ s}$
50	6	3	61	$2^{17.20}$	14	$2^{-5.41}$	$2^{13.76}$	$2^{3.44} \text{ s} \approx 10.9 \text{ s}$
51	6	4	62	$2^{18.41}$	2	$2^{-18.43}$	$2^{18.64}$	$\approx 0.9 \text{ s}$
52	6	5	63	$2^{19.12}$	17	$2^{-7.26}$	$2^{15.22}$	$2^{3.89} \text{ s} \approx 14.9 \text{ s}$
53	6	3	60	$2^{21.13}$	3	$2^{-20.12}$	$2^{18.49}$	$2^{2.64} \text{ s} \approx 6.2 \text{ s}$
55	6	4	61	$2^{22.78}$	16	$2^{-17.58}$	$2^{18.46}$	$2^{4.32} \text{ s} \approx 20.0 \text{ s}$
57	6	3	59	$2^{24.71}$	8	$2^{-17.37}$	$2^{18.40}$	$2^{6.31} \text{ s} \approx 1.3 \text{ min}$
58	6	2	57	$2^{25.63}$	20	$2^{-15.15}$	$2^{18.82}$	$2^{6.81} \text{ s} \approx 1.9 \text{ min}$
59	6	4	60	$2^{26.71}$	15	$2^{-17.93}$	$2^{18.47}$	$2^{8.24} \text{ s} \approx 5.1 \text{ min}$
60	6	3	58	$2^{28.01}$	10	$2^{-19.41}$	$2^{18.78}$	$2^{9.24} \text{ s} \approx 10.0 \text{ min}$
63	6	2	55	$2^{31.10}$	4	$2^{-19.78}$	$2^{19.09}$	$2^{12.01} \text{ s} \approx 1.1 \text{ h}$
65	6	2	54	$2^{33.56}$	14	$2^{-17.80}$	$2^{18.66}$	$2^{14.90} \text{ s} \approx 8.5 \text{ h}$
66	6	3	56	$2^{33.90}$	15	$2^{-7.06}$	$2^{15.10}$	$2^{18.80} \text{ s} \approx 5.3 \text{ d}$
68	6	2	53	$2^{35.85}$	2	$2^{-19.74}$	$2^{19.45}$	$2^{16.40} \text{ s} \approx 1.0 \text{ d}$
69	6	4	57	$2^{36.65}$	16	$2^{-17.56}$	$2^{18.82}$	$2^{17.83} \text{ s} \approx 2.7 \text{ d}$
70	8	5	255	$2^{26.68}$	28	$2^{-9.28}$	$2^{11.82}$	$2^{14.86} \text{ s} \approx 8.3 \text{ h}$
71	6	6	60	$2^{38.13}$	9	$2^{-17.23}$	$2^{18.79}$	$2^{19.34} \text{ s} \approx 7.7 \text{ d}$
72	7	5	125	$2^{34.26}$	2	$2^{-23.77}$	$2^{16.22}$	$2^{18.04} \text{ s} \approx 3.1 \text{ d}$
73	7	6	126	$2^{35.61}$	1	$2^{-23.92}$	$2^{16.09}$	$2^{19.52} \text{ s} \approx 8.7 \text{ d}$
74	7	8	128	$2^{36.20}$	26	$2^{-6.95}$	$2^{10.79}$	$2^{25.41} \text{ s} \approx 1.41 \text{ yr}$
76	6	7	60	$2^{43.91}$	3	$2^{-18.87}$	$2^{19.05}$	$2^{24.85} \text{ s} \approx 0.96 \text{ yr}$
77	7	5	124	$2^{39.23}$	6	$2^{-19.15}$	$2^{15.90}$	$2^{23.33} \text{ s} \approx 4.0 \text{ mo}$
78	6	8	61	$2^{45.78}$	5	$2^{-18.29}$	$2^{18.48}$	$2^{27.31} \text{ s} \approx 5.26 \text{ yr}$
79	7	6	125	$2^{41.00}$	5	$2^{-19.46}$	$2^{15.92}$	$2^{25.08} \text{ s} \approx 1.12 \text{ yr}$
80	7	7	126	$2^{42.39}$	2	$2^{-21.02}$	$2^{16.02}$	$2^{26.36} \text{ s} \approx 2.74 \text{ yr}$
81	7	8	127	$2^{43.20}$	5	$2^{-18.43}$	$2^{15.38}$	$2^{27.82} \text{ s} \approx 7.52 \text{ yr}$
82	6	8	60	$2^{49.72}$	6	$2^{-18.92}$	$2^{18.59}$	$2^{31.13} \text{ s} \approx 74.34 \text{ yr}$
83	8	5	253	$2^{40.08}$	1	$2^{-19.94}$	$2^{13.22}$	$2^{26.86} \text{ s} \approx 3.86 \text{ yr}$
84	8	6	254	$2^{41.42}$	28	$2^{-10.64}$	$2^{12.61}$	$2^{28.80} \text{ s} \approx 14.86 \text{ yr}$
85	8	8	256	$2^{42.01}$	25	$2^{-10.31}$	$2^{9.98}$	$2^{32.03} \text{ s} \approx 138.8 \text{ yr}$
86	7	5	122	$2^{48.22}$	1	$2^{-19.23}$	$2^{16.44}$	$2^{31.78} \text{ s} \approx 117.0 \text{ yr}$
87	7	8	126	$2^{49.19}$	4	$2^{-18.28}$	$2^{15.79}$	$2^{33.40} \text{ s} \approx 359.4 \text{ yr}$
88	7	9	127	$2^{50.03}$	26	$2^{-6.78}$	$2^{12.13}$	$2^{37.90} \text{ s} \approx 8,155 \text{ yr}$
89	8	5	252	$2^{46.06}$	1	$2^{-21.19}$	$2^{13.23}$	$2^{32.83} \text{ s} \approx 242.2 \text{ yr}$
90	7	5	121	$2^{52.34}$	4	$2^{-17.60}$	$2^{16.20}$	$2^{36.14} \text{ s} \approx 2,399 \text{ yr}$
91	8	6	253	$2^{47.82}$	18	$2^{-13.32}$	$2^{13.02}$	$2^{34.79} \text{ s} \approx 944.4 \text{ yr}$
92	8	7	254	$2^{49.19}$	4	$2^{-19.14}$	$2^{13.10}$	$2^{36.10} \text{ s} \approx 2,329 \text{ yr}$
93	8	8	255	$2^{50.01}$	29	$2^{-9.06}$	$2^{11.71}$	$2^{38.30} \text{ s} \approx 10,696 \text{ yr}$
94	7	5	120	$2^{56.26}$	7	$2^{-18.34}$	$2^{16.23}$	$2^{40.03} \text{ s} \approx 35,557 \text{ yr}$
95	7	7	123	$2^{57.38}$	2	$2^{-23.08}$	$2^{16.39}$	$2^{40.99} \text{ s} \approx 69,152 \text{ yr}$
96	9	8	512	$2^{47.83}$	4	$2^{-20.24}$	$2^{8.35}$	$2^{39.48} \text{ s} \approx 24,359 \text{ yr}$
97	7	8	124	$2^{59.55}$	12	$2^{-15.02}$	$2^{16.19}$	$2^{43.36} \text{ s} \approx 358,402 \text{ yr}$
99	7	9	125	$2^{61.42}$	5	$2^{-19.82}$	$2^{16.09}$	$2^{45.32} \text{ s} \approx 1,395,262 \text{ yr}$

Table 2. Overview of the smaller parameter sets from the *TII McEliece Challenges* together with attack complexity estimates using our optimized key-search software *including* the use of extended Goppa codes. The empirical throughput (private-key guesses per core and second) was measured on an AMD EPYC 9754 processor (Zen 4c microarchitecture) running at 2.25 GHz with hyperthreading disabled.

The required number of guesses was estimated based on [Heuristic 14](#). The columns “ $\# \mathcal{F}$ ” and “ $\approx \Pr[\mathcal{F} \mapsto \text{true}]$ ” refer to the length of the used fast filter $\mathcal{F}$, not including the unpunctured hull enumerator itself, and the empirical probability for a random code to pass the filter $\mathcal{F}$, respectively. All filters were constructed by running [Algorithm 8](#) with $K = 100,000$. Note that no attempt was made to optimize the estimated total duration of the attack by adjusting the length of the filter.

instance	m	t	n	$\approx \# \text{ guesses}$	$\# \mathcal{F}$	$\approx \Pr[\mathcal{F} \mapsto \text{true}]$	guesses/(core · s)	$\approx \text{core time}$
41	5	2	27	$2^{9.49}$	3	$2^{-8.49}$	$2^{17.73}$	$\approx 0.0 \text{ s}$
43	5	2	26	$2^{11.71}$	5	$2^{-4.34}$	$2^{14.83}$	$\approx 0.1 \text{ s}$
44	6	2	61	$2^{5.81}$	2	$2^{-5.81}$	$2^{11.93}$	$\approx 0.0 \text{ s}$
45	6	3	62	$2^{6.85}$	5	$2^{-3.53}$	$2^{11.42}$	$\approx 0.0 \text{ s}$
46	6	4	63	$2^{7.44}$	1	$2^{-7.44}$	$2^{13.70}$	$\approx 0.0 \text{ s}$
47	5	2	23	$2^{16.83}$	11	$2^{-8.20}$	$2^{18.75}$	$\approx 0.3 \text{ s}$
48	6	2	60	$2^{9.76}$	1	$2^{-9.76}$	$2^{15.73}$	$\approx 0.0 \text{ s}$
49	5	2	21	$2^{19.16}$	10	$2^{-5.53}$	$2^{16.51}$	$2^{2.65} \text{ s} \approx 6.3 \text{ s}$
50	6	3	61	$2^{11.24}$	4	$2^{-11.25}$	$2^{16.75}$	$\approx 0.0 \text{ s}$
51	6	4	62	$2^{12.44}$	1	$2^{-12.43}$	$2^{17.28}$	$\approx 0.0 \text{ s}$
52	6	5	63	$2^{13.12}$	4	$2^{-13.12}$	$2^{17.05}$	$\approx 0.1 \text{ s}$
53	6	3	60	$2^{15.20}$	3	$2^{-15.19}$	$2^{17.81}$	$\approx 0.2 \text{ s}$
55	6	4	61	$2^{16.83}$	7	$2^{-16.84}$	$2^{18.38}$	$\approx 0.3 \text{ s}$
57	6	3	59	$2^{18.81}$	12	$2^{-18.83}$	$2^{18.43}$	$2^{0.37} \text{ s} \approx 1.3 \text{ s}$
58	6	2	57	$2^{19.77}$	19	$2^{-14.07}$	$2^{18.50}$	$2^{1.27} \text{ s} \approx 2.4 \text{ s}$
59	6	4	60	$2^{20.78}$	2	$2^{-20.78}$	$2^{17.99}$	$2^{2.79} \text{ s} \approx 6.9 \text{ s}$
60	6	3	58	$2^{22.13}$	3	$2^{-19.80}$	$2^{18.81}$	$2^{3.32} \text{ s} \approx 10.0 \text{ s}$
63	6	2	55	$2^{25.29}$	1	$2^{-19.18}$	$2^{19.31}$	$2^{5.98} \text{ s} \approx 1.1 \text{ min}$
65	6	2	54	$2^{27.77}$	3	$2^{-19.66}$	$2^{18.92}$	$2^{8.85} \text{ s} \approx 7.7 \text{ min}$
66	6	3	56	$2^{28.06}$	15	$2^{-5.70}$	$2^{14.23}$	$2^{13.84} \text{ s} \approx 4.1 \text{ h}$
68	6	2	53	$2^{30.10}$	2	$2^{-21.57}$	$2^{19.12}$	$2^{10.97} \text{ s} \approx 33.5 \text{ min}$
69	6	4	57	$2^{30.79}$	11	$2^{-18.77}$	$2^{18.56}$	$2^{12.23} \text{ s} \approx 1.3 \text{ h}$
70	8	5	255	$2^{18.68}$	4	$2^{-17.74}$	$2^{12.54}$	$2^{6.14} \text{ s} \approx 1.2 \text{ min}$
71	6	6	60	$2^{32.20}$	4	$2^{-20.23}$	$2^{18.58}$	$2^{13.61} \text{ s} \approx 3.5 \text{ h}$
72	7	5	125	$2^{27.29}$	2	$2^{-18.00}$	$2^{16.14}$	$2^{11.15} \text{ s} \approx 37.8 \text{ min}$
73	7	6	126	$2^{28.62}$	0	$2^{-21.82}$	$2^{15.97}$	$2^{12.65} \text{ s} \approx 1.8 \text{ h}$
74	7	8	128	$2^{29.19}$	19	$2^{-8.48}$	$2^{11.10}$	$2^{18.09} \text{ s} \approx 3.2 \text{ d}$
76	6	7	60	$2^{37.98}$	4	$2^{-19.72}$	$2^{18.24}$	$2^{19.73} \text{ s} \approx 10.1 \text{ d}$
77	7	5	124	$2^{32.26}$	6	$2^{-18.96}$	$2^{15.84}$	$2^{16.43} \text{ s} \approx 1.0 \text{ d}$
78	6	8	61	$2^{39.83}$	7	$2^{-20.08}$	$2^{18.49}$	$2^{21.34} \text{ s} \approx 1.0 \text{ mo}$
79	7	6	125	$2^{34.02}$	5	$2^{-19.19}$	$2^{15.74}$	$2^{18.28} \text{ s} \approx 3.7 \text{ d}$
80	7	7	126	$2^{35.40}$	1	$2^{-20.74}$	$2^{16.01}$	$2^{19.39} \text{ s} \approx 7.9 \text{ d}$
81	7	8	127	$2^{36.20}$	6	$2^{-19.75}$	$2^{15.34}$	$2^{20.86} \text{ s} \approx 22.1 \text{ d}$
82	6	8	60	$2^{43.78}$	3	$2^{-22.40}$	$2^{18.31}$	$2^{25.48} \text{ s} \approx 1.48 \text{ yr}$
83	8	5	253	$2^{32.09}$	1	$2^{-25.90}$	$2^{13.03}$	$2^{19.06} \text{ s} \approx 6.3 \text{ d}$
84	8	6	254	$2^{33.42}$	25	$2^{-9.19}$	$2^{11.92}$	$2^{21.50} \text{ s} \approx 1.1 \text{ mo}$
85	8	8	256	$2^{34.00}$	9	$2^{-17.69}$	$2^{10.03}$	$2^{23.97} \text{ s} \approx 6.2 \text{ mo}$
86	7	5	122	$2^{41.28}$	1	$2^{-20.37}$	$2^{16.34}$	$2^{24.94} \text{ s} \approx 1.02 \text{ yr}$
87	7	8	126	$2^{42.20}$	2	$2^{-21.41}$	$2^{15.64}$	$2^{26.56} \text{ s} \approx 3.14 \text{ yr}$
88	7	9	127	$2^{43.03}$	19	$2^{-8.56}$	$2^{13.75}$	$2^{29.28} \text{ s} \approx 20.69 \text{ yr}$
89	8	5	252	$2^{38.08}$	2	$2^{-22.05}$	$2^{13.14}$	$2^{24.94} \text{ s} \approx 1.02 \text{ yr}$
90	7	5	121	$2^{45.41}$	2	$2^{-20.77}$	$2^{16.17}$	$2^{29.24} \text{ s} \approx 20.12 \text{ yr}$
91	8	6	253	$2^{39.83}$	4	$2^{-19.07}$	$2^{12.91}$	$2^{26.92} \text{ s} \approx 4.03 \text{ yr}$
92	8	7	254	$2^{41.20}$	1	$2^{-20.46}$	$2^{13.07}$	$2^{28.12} \text{ s} \approx 9.27 \text{ yr}$
93	8	8	255	$2^{42.01}$	3	$2^{-18.10}$	$2^{12.68}$	$2^{29.32} \text{ s} \approx 21.27 \text{ yr}$
94	7	5	120	$2^{49.34}$	10	$2^{-18.05}$	$2^{15.97}$	$2^{33.37} \text{ s} \approx 352.6 \text{ yr}$
95	7	7	123	$2^{50.42}$	2	$2^{-19.79}$	$2^{16.17}$	$2^{34.26} \text{ s} \approx 650.0 \text{ yr}$
96	9	8	512	$2^{38.83}$	1	$2^{-19.91}$	$2^{8.34}$	$2^{30.49} \text{ s} \approx 47.93 \text{ yr}$
97	7	8	124	$2^{52.58}$	17	$2^{-10.17}$	$2^{15.03}$	$2^{37.55} \text{ s} \approx 6,385 \text{ yr}$
99	7	9	125	$2^{54.44}$	2	$2^{-24.81}$	$2^{16.06}$	$2^{38.38} \text{ s} \approx 11,305 \text{ yr}$

References

- [ADKF70] Владимир Л. Арлазаров, Ефим А. Диниц, Михаил А. Кронрод and Игорь А. Фараджев. “Об экономном построении транзитивного замыкания ориентированного графа”. In: *Доклады Академии Наук СССР* 194 (3 1970), pp. 487–488.
- [Ber00] Thierry P. Berger. “On the Cyclicity of Goppa Codes, Parity-Check Subcodes of Goppa Codes, and Extended Goppa Codes”. In: *Finite Fields and Their Applications* 6.3 (2000), pp. 255–281. DOI: [10.1006/ffta.2000.0277](https://doi.org/10.1006/ffta.2000.0277).
- [Ber24] Daniel J. Bernstein. “Understanding binary-Goppa decoding”. In: *IACR Communications in Cryptology* 1.1 (2024). DOI: [10.62056/angy4fe-3](https://doi.org/10.62056/angy4fe-3).
- [BLP08] Daniel J. Bernstein, Tanja Lange and Christiane Peters. “Attacking and Defending the McEliece Cryptosystem”. In: *PQCrypto 2008*. Vol. 5299. Lecture Notes in Computer Science. Springer, 2008, pp. 31–46. URL: <https://ia.cr/2018/318>.
- [CME22r] Martin R. Albrecht, Daniel J. Bernstein, Tung Chou, Carlos Cid, Jan Gilcher, Tanja Lange, Varun Maram, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Kenneth G. Paterson, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, Jakub Szefer, Cen Jung Tjhai, Martin Tomlinson and Wen Wang. *Classic McEliece: conservative code-based cryptography. Design rationale*. Tech. rep. 2022. URL: <https://classic.mceliece.org/mceliece-rationale-20221023.pdf>.
- [Gib91] J. Keith Gibson. “Equivalent Goppa Codes and Trapdoors to McEliece’s Public Key Cryptosystem”. In: *EUROCRYPT 1991*. Vol. 547. Lecture Notes in Computer Science. Springer, 1991, pp. 517–521. DOI: [10.1007/3-540-46416-6_46](https://doi.org/10.1007/3-540-46416-6_46).
- [LS01] Pierre Loidreau and Nicolas Sendrier. “Weak keys in the McEliece public-key cryptosystem”. In: *IEEE Transactions on Information Theory* 47.3 (2001), pp. 1207–1211.
- [M4RI] Martin Albrecht and Gregory Bard. *The M4RI Library*. Version 20250128. The M4RI Team. 2025. URL: <https://bitbucket.org/malb/m4ri>.
- [McE78] Robert J. McEliece. *A public-key cryptosystem based on algebraic coding theory*. The Deep Space Network Progress Report 42-44. 1978, pp. 114–116. URL: https://ipnpr.jpl.nasa.gov/progress_report/42-44/44N.PDF.

- [Mor79] Oscar Moreno. “Symmetries of binary Goppa codes (Corresp.)” In: *IEEE Transactions on Information Theory* 25.5 (1979), pp. 609–612. DOI: [10.1109/tit.1979.1056089](https://doi.org/10.1109/tit.1979.1056089).
- [MS81] Florence Jessie MacWilliams and Neil James Alexander Sloane. *The Theory of Error-Correcting Codes*. 3rd ed. North-Holland Publishing Company, 1981.
- [OS09] Raphael Overbeck and Nicolas Sendrier. “Code-based cryptography”. In: *Post-Quantum Cryptography*. Ed. by Daniel J. Bernstein, Johannes Buchmann and Erik Dahmen. Springer, 2009, pp. 95–145.
- [Pat75] Nicholas J. Patterson. “The algebraic decoding of Goppa codes”. In: *IEEE Transactions on Information Theory* 21.2 (1975), pp. 203–207. DOI: [10.1109/TIT.1975.1055350](https://doi.org/10.1109/TIT.1975.1055350).
- [Sen00] Nicolas Sendrier. “Finding the permutation between equivalent linear codes: The support splitting algorithm”. In: *IEEE Transactions on Information Theory* 46.4 (2000), pp. 1193–1203. URL: <https://inria.hal.science/inria-00073037/document>.
- [Sen97] Nicolas Sendrier. “On the Dimension of the Hull”. In: *SIAM Journal on Discrete Mathematics* 10.2 (1997), pp. 282–293. URL: <https://inria.hal.science/inria-00074009/document>.

A Proofs

A.1 Proof of Lemma 1

Proof. First, note that $\alpha_{n+1} = \infty$, which implies that $(f/g)(\alpha_{n+1}) = 0$ whenever $\deg(f) < \deg(g) = t$. Call this property “ $\dagger$ ”.

Inclusion “ $\subseteq$ ”: Let $\widehat{c} = (c_1, \dots, c_{n+1}) \in \widehat{\Gamma}(g, \widehat{L})$ and write $c = (c_1, \dots, c_n) \in \mathbb{F}_q^n$. By assumption, we have $\dagger$: $\forall f \in \mathbb{F}_q[x]_{\leq t}$. $\sum_{i=1}^{n+1} c_i(f/g)(\alpha_i) = 0$, which implies $\forall f \in \mathbb{F}_q[x]_{< t}$. $\sum_{i=1}^n c_i(f/g)(\alpha_i) = 0$ thanks to $\dagger$, hence $c = (c_1, \dots, c_n) \in \Gamma(g, L)$. It remains to show that $c_{n+1} = -c_1 - \dots - c_n$. This is immediate by instantiating $\dagger$ with $f = g$.

Inclusion “ $\supseteq$ ”: Let $c = (c_1, \dots, c_n) \in \Gamma(g, L)$ and set $\widehat{c} = (c_1, \dots, c_n, c_{n+1}) \in \mathbb{F}_q^{n+1}$ where $c_{n+1} = -c_1 - \dots - c_n$. Suppose given some $f \in \mathbb{F}_q[x]_{\leq t}$. If $\deg(f) < t$, then $\sum_{i=1}^n c_i(f/g)(\alpha_i) = 0$ by assumption, which (thanks to $\dagger$) is equivalent to the desired statement $\sum_{i=1}^{n+1} c_i(f/g)(\alpha_i) = 0$. Now suppose $\deg(f) = t$: Setting $h = \text{lc}(f)$, we get $f = gh + r$ for some $r \in \mathbb{F}_q[x]$ with $\deg(r) < t$. From the implication we just proved (now applied to r instead of f), we get $\sum_{i=1}^{n+1} c_i(r/g)(\alpha_i) = 0$. Hence

$$\sum_{i=1}^{n+1} c_i(f/g)(\alpha_i) = \sum_{i=1}^{n+1} c_i(h + r/g)(\alpha_i) = h \underbrace{\sum_{i=1}^{n+1} c_i}_{=0} + \underbrace{\sum_{i=1}^{n+1} c_i(r/g)(\alpha_i)}_{=0} = 0,$$

which was to be demonstrated. $\square$

A.2 Proof of Lemma 4

Proof. Since φ, φ' are field automorphisms, they leave $0, 1 \in \mathbb{F}_q$ invariant. Hence, substituting $x = 0$ into $Ax^\varphi + B = A'x^{\varphi'} + B'$ yields $B = B'$, and substituting $x = 1$ then shows $A + B = A' + B$, hence $A = A'$. Finally, if $Ax^\varphi + B = A + x^{\varphi'} + B$ for all $x \in \mathbb{F}_q$, then $x^\varphi = x^{\varphi'}$ for all $x \in \mathbb{F}_q$, therefore $\varphi = \varphi'$ as maps. $\square$

A.3 Proof of Lemma 8

Proof. Let $\sigma \in \text{Gal}(\overline{\mathbb{F}_q}/\mathbb{F}_q)$. Since h is defined over $\mathbb{F}_q$, there is a permutation π of the set $\{1, \dots, d\}$ such that $\varrho_j^\sigma = \varrho_{\pi(j)}$ for all j . Utilizing the fact that $\text{Aut}(\overline{\mathbb{F}_q})$ is abelian and hence $\Phi\sigma = \sigma\Phi$, we therefore get

$$(A\varrho_j^\Phi + B)^\sigma = A\varrho_j^{\Phi\sigma} + B = A\varrho_{\pi(j)}^\Phi + B.$$

This shows that σ acts as a permutation on the elements $A\varrho_j^\Phi + B$, which by definition make up the roots of $\tau * h$, and therefore $\tau * h$ must be defined over $\mathbb{F}_q$.

It remains to show that the choice of lift Φ does not matter: This is because for all $\Phi, \Phi' \in \text{Aut}(\overline{\mathbb{F}_q})$ which restrict to φ in $\text{Aut}(\mathbb{F}_q)$, we have $\Phi^{-1}\Phi' \in \text{Gal}(\overline{\mathbb{F}_q}/\mathbb{F}_q)$. Let π again denote a permutation such that $\varrho_j^{\Phi^{-1}\Phi'} = \varrho_{\pi(j)}$ for all j . Then

$$\varrho_j^{\Phi'} = \varrho_j^{\Phi\Phi^{-1}\Phi'} = \varrho_{\pi(j)}^\Phi,$$

thus replacing Φ by Φ' merely induces a permutation on the elements $A\varrho_j^\Phi + B$, which leaves the polynomial $\tau * h$ invariant. $\square$

A.4 Proof of Lemma 12

Proof (sketch). For brevity, we give a proof sketch for the following (stronger) claim, morally similar to Lemma 4: The group $\text{PTL}(q)$ is in bijection with the set

$$S = \{(A, B, C, D, \varphi) \in \mathbb{F}_q^4 \times \text{Aut}(\mathbb{F}_q) \mid \det \begin{pmatrix} A & B \\ C & D \end{pmatrix} \neq 0\}$$

modulo the equivalence relation

$$(A, B, C, D, \varphi) \sim (A', B', C', D', \varphi') \iff \varphi = \varphi' \text{ and } \exists \lambda \in \mathbb{F}_q^\times. \begin{pmatrix} A' & B' \\ C' & D' \end{pmatrix} = \lambda \begin{pmatrix} A & B \\ C & D \end{pmatrix},$$

by identifying the class of (A, B, C, D, φ) with the map $x \mapsto \frac{Ax^\varphi + B}{Cx^\varphi + D}$ in $\text{Sym}(\mathbb{P}^1 \mathbb{F}_q)$.

This mapping from $S/\sim$ to $\text{PTL}(q)$ is obviously well-defined and surjective. Moreover, after defining a group structure on $S/\sim$ using the *symbolic* composition of the associated functions in $\text{PTL}(q)$, this map becomes a group homomorphism. To prove injectivity, let $(A, B, C, D, \varphi) \in S$ such that $\tau: x \mapsto \frac{Ax^\varphi + B}{Cx^\varphi + D} \in \text{PTL}(q)$ is the identity map on $\mathbb{P}^1 \mathbb{F}_q$. Then, substituting the points $0, 1, \infty$ into $\tau(x) = x$:

$$\begin{aligned} \infty &= \tau(\infty) = A/C; \\ 0 &= \tau(0) = B/D; \\ 1 &= \tau(1) = (A + B)/(C + D). \end{aligned}$$

Together, these imply $B = C = 0$ and $A = D$, hence $\tau: x \mapsto x^\varphi$. Finally, $x^\varphi = x$ for all $x \in \mathbb{F}_q$ implies $\varphi = \text{id} \in \text{Aut}(\mathbb{F}_q)$. In summary, $(A, B, C, D, \varphi) = (\lambda, 0, 0, \lambda, \text{id})$ for some $\lambda \in \mathbb{F}_q^\times$. $\square$

A.5 Proof of Lemma 19

Proof. Let M_{ij} denote the coefficient of M in the i^{th} row and j^{th} column. The entry in the i^{th} row and j^{th} column of M^2 thus equals $c = \sum_{k=1}^n M_{ik} M_{kj}$. Since all entries of M below the diagonal are zero, this sum shortens to $c = \sum_{k=i}^j M_{ik} M_{kj}$. For each k , if $M_{kk} = 1$, then all M_{ik} with $i < k$ lie above a pivot and therefore must be zero, and if $M_{kk} = 0$, then the entire k^{th} row, thus all M_{kj} , must be zero. Since in both cases, it follows that $M_{ik} M_{kj} = 0$ unless $k = i$, we are left with $c = M_{ii} M_{ij}$. Now, if $M_{ii} = 1$, then $c = M_{ij}$ as claimed. If on the other hand $M_{ii} = 0$, then the entire i^{th} row is zero and it follows that $c = M_{ij} = 0$.

From $M^2 = M$ we get $M \cdot (\mathbb{1} - M) = 0$, hence the columns of $\mathbb{1} - M$ are indeed contained in the right kernel of M . The matrix $\mathbb{1} - M$ is upper triangular with every nonzero column having a 1 on the diagonal, hence the nonzero columns are linearly independent. Finally, from the rank-nullity theorem, the nonzero columns of $\mathbb{1} - M$ must form a basis of the right kernel since they correspond precisely to the zero rows of M . $\square$

B Example: Boolean circuit for linear algebra

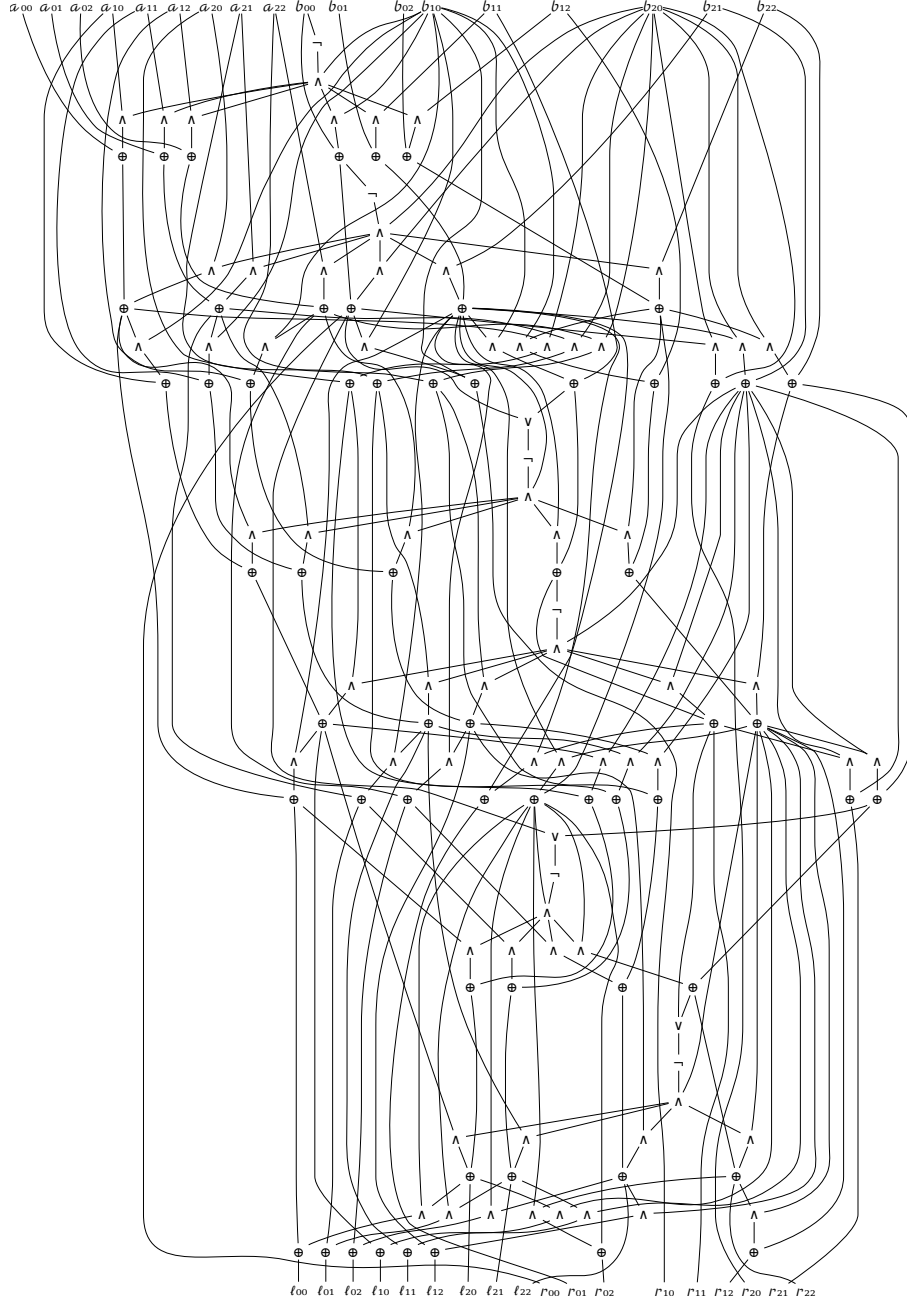


Figure 3. Logic circuit constructed by [Algorithm 2](#) for $n=3$.